

Phenix Documentation (Miscellaneous)

11 documentation pages

Generated from local Phenix documentation

Contents

1. How to rebuild PHENIX from an existing installation
2. How to build PHENIX on Windows
3. Using Coot with PHENIX
4. Dictionary of common terms used in PHENIX
5. Building the PHENIX documentation
6. PHENIX Examples
7. About Phenix-supported file formats
8. Main PHENIX Modules
9. The phenix graphical interface
10. Overview of Phenix Programs
11. Using PyMOL with PHENIX

How to rebuild PHENIX from an existing installation

ManualPhenixBuild.html

Contents

- Introduction Prerequisites on Linux Prerequisites on MacOSX
- Prerequisites on Linux
- Prerequisites on MacOSX
- Build procedure
- Windows Build

Introduction

Phenix is build nightly through an automated bootstrap mechanism. This procedure is very lengthy and only available to authorised Phenix developers. If you have a recent existing installation of Phenix and wish to recompile the sources on your own machine a quicker procedure exists. You do not have to be a Phenix developer either. This document describes how to build Phenix from the sources in your installation without using bootstrap.py. It applies to both Linux and MacOSX. Links to Windows build is included.

Prerequisites on Linux

To build PHENIX from an existing installation with sources the following is required:

gcc and g++ compilers OpenGL headers (on Ubuntu fetch with `sudo apt-get install libglu1-mesa-dev freeglut3-dev mesa-common-dev`)

- gcc and g++ compilers
- OpenGL headers (on Ubuntu fetch with `sudo apt-get install libglu1-mesa-dev freeglut3-dev mesa-common-dev`)

Prerequisites on MacOSX

To build PHENIX from an existing installation with sources the following is required:

Xcode development tools

- Xcode development tools

Build procedure

Assuming your installation is located in `/path/to/my/phenix/` change directory from a Bash shell to `/path/to/my/phenix/build`. Then from that Bash shell and in that directory run the commands:

```
shopt -s extglob
rm -- !(config_modules.sh)
```

This has the effect of deleting all files and directories from the build directory except for the `config_modules.sh` file. Now within the `/path/to/my/phenix/build` directory run the command:

```
./config_modules.sh
```

This will prepare the build of Phenix using the python and necessary base components shipped with your Phenix installation. Then run the commands:

```
source setpaths.sh  
libtbx.scons -j nproc
```

where nproc is the number of cpu cores you want to use for the build. The build should take less than an hour on a modern multicore PC.

Windows Build

The above procedure is much the same for building on Windows systems. [Click here](#) for further details.

How to build PHENIX on Windows

WindowsBuild.html

Contents

- Introduction
- Necessary tweaks of the Windows PC Automatically run script for every command prompt Run a script once at every logon
- Automatically run script for every command prompt
- Run a script once at every logon
- Necessary prerequisites Visual C++ 9.0 OpenSSH tools Makensis
- Visual C++ 9.0
- OpenSSH tools
- Makensis
- Bundling base components together as zip files
- Manual build From an existing PHENIX installation From bootstrap.py
- From an existing PHENIX installation
- From bootstrap.py
- Automated build with Buildbot Issue defining environment variables for the compiler Setting up a PC as a Buildbot slave
- Issue defining environment variables for the compiler
- Setting up a PC as a Buildbot slave
- Creating an executable Windows installer program
- Important tweaks for shortening duration of builds

Introduction

This document describes how to build Phenix from sources on Windows 7 or more recent versions. Given that Windows does not have all the necessary tools for building Phenix, how to set up these tools is covered here in some detail.

A typical PHENIX installation takes up the order of 5Gb disk space. If you build PHENIX from scratch including an installer this will be of the order of 15Gb. Throughout this document the "command prompt" (archaically termed the "DOS prompt" by some) refers to the commandline interpreter program, cmd.exe, which serves the same purpose as bash or tcsh on Unix. On the Startup menu it is located in the "Accessories" group.

Necessary tweaks of the Windows PC

Automatically run script for every command prompt

On Unix a startup script such as .bashrc or .tcshrc are convenient for initiating commonly used environment variables for a user. Windows has a different technique for achieving the same for the command prompt. In the registry editor, regedit.exe, locate the AutoRun registry key:

HKEY_CURRENT_USER\Software\Microsoft\Command Processor\AutoRun

If there is no AutoRun key present under the "Command Processor" key then create a new one of type string. The value of this key can be set to the full path of a cmd script you have written that does something such as defining environment variables for that command prompt. This script can have any name but we will call it ntshell.cmd and store it in path pointed to by the %HOMEDRIVE%%HOMEPATH% environment variables.

Warning

Entering wrong values in the registry or accidentally deleting values may render Windows bricked. Always ensure you have a recent "System Restore" point in your Windows backup. This will allow you to boot Windows to "Last known good configuration" if the registry gets corrupted.

Alternatively one can also define environment variables through the Control Panel\All Control Panel Items\User Accounts and click on "Change my environment variables". But this method is not feasible if there are many variables or some variable values are deduced on the fly from scripts. This is the case when initialising environment variables for the compiler.

Run a script once at every logon

Say you have a script in the directory pointed to by the %HOMEDRIVE%%HOMEPATH% environment variables named logon.cmd or whatever. To make this script run once every time you log on to your account, open the Task Scheduler in the "Administrative tools" group in Control Panel, click "Create Task", set "Triggers" to "At logon" and set "Action" to the full path of the logon.cmd file. This script will then run next time you log on.

Necessary prerequisites

To build PHENIX from scratch and create a binary Windows installer the following tools are required:

Python 2.7 (any version) makensis from Nullsoft Microsoft Visual C++ 9.0 / Microsoft Visual Studio 2008 TortoiseSVN with command line tools TortoiseGit (optionally) Git mtee.exe (optionally for duplicating stdout to file and console)

- Python 2.7 (any version)
- makensis from Nullsoft
- Microsoft Visual C++ 9.0 / Microsoft Visual Studio 2008
- TortoiseSVN with command line tools
- TortoiseGit (optionally)
- Git
- mtee.exe (optionally for duplicating stdout to file and console)

To build PHENIX from an existing installation with sources the following is required:

Microsoft Visual C++ 9.0 / Microsoft Visual Studio 2008

- Microsoft Visual C++ 9.0 / Microsoft Visual Studio 2008

Visual C++ 9.0

If you have not purchased Microsoft Visual Studio 2008 the compiler is available for free in the Windows 7 SDK from the Microsoft website: <https://www.microsoft.com/en-gb/download/details.aspx?id=3138&fa43d42b-25b5-4a42-fe9b-1634f450f5ee=True> . It should be titled "Microsoft Windows SDK for Windows 7 and .NET

Framework 3.5 SP1". Rather than doing the automated installation from the website it may be convenient to download the DVD iso image of the SDK and then do the installation from that. The 32 bit version is named GRMSDK_EN_DVD.iso whereas the 64 bit version is named GRMSDKX_EN_DVD.iso. There are newer versions of this SDK for Windows 7. But as these use more recent C-runtime libraries compiled programs may not be binary compatible with Python 2.7 which is compiled with Visual C++ 9.0. You would still be able to build individual statically linked executables with those newer compilers. But whether or not compiled python modules can run without crashing using a newer runtime is questionable. After installing the compiler it can be invoked once environment variables have been set by calling scripts such as vcvars32.bat or vcvars64.bat from a command prompt or in a build script prior to compiling. These files live in the folder

```
C:\Program Files (x86)\Microsoft Visual Studio 9.0\VC\bin
```

OpenSSH tools

The OpenSSH tools used for the Windows build are the ones that come with Git for Windows. If Git has been installed in

```
C:\Program Files\Git
```

then OpenSSH tools live in the directory

```
C:\Program Files\Git\usr\bin
```

which you must add to the %PATH% environment variable. To have the build run without interrupts the private keys matching your public keys on the remote svn or git repositories must be loaded prior to the build with ssh-agent. Create a .ssh directory in the directory pointed to by the %USERPROFILE% environment variable and store your private keys there. Then add the lines

```
call "C:\Program Files\Git\cmd\start-ssh-agent.cmd"
SETX SSH_AUTH_SOCK "%SSH_AUTH_SOCK%"
SETX SSH_AGENT_PID "%SSH_AGENT_PID%"
```

to your logon.cmd file. When you log on it will pick up the private keys in your %USERPROFILE%\ssh folder. If keys require password you will be prompted for them when you logon to the PC. If a key, say mykey_rsa, is not picked up it can be loaded explicitly by adding the line

```
ssh-add mykey_rsa
```

just below the above 3 lines.

Makensis

Makensis from Nullsoft is only required for creating an executable Windows installer package for deployment of PHENIX. Bundled up zip files of an installation is always created.

Bundling base components together as zip files

The Windows build does not need to compile Python or the HDF5 library from scratch. Instead these two components are bundled as separate zip files which bootstrap.py will download from http://cci.lbl.gov/cctbx_dependencies during the first stages of the build. The Python bundle has had all the necessary 3rd party modules added to it for running PHENIX on a PC. Click [here](#) for details on how these bundles were created or on how to update this Python bundle with additional modules.

Manual build

From an existing PHENIX installation

If you want to build PHENIX from an existing installation on Windows where the sources have also been installed then do the following:

From the Start menu open a PHENIX Command Prompt. Run vcvars32.bat or vcvars64.bat depending on the platform. Change directory to %PHENIX%\build. If compiling for the first time delete all files and directories except config_modules.cmd. Run config_modules.cmd. Run libtbx.scons -j <nproc>.

- From the Start menu open a PHENIX Command Prompt.
- Run vcvars32.bat or vcvars64.bat depending on the platform.
- Change directory to %PHENIX%\build. If compiling for the first time delete all files and directories except config_modules.cmd.

Change directory to %PHENIX%\build.

If compiling for the first time delete all files and directories except config_modules.cmd.

- If compiling for the first time delete all files and directories except config_modules.cmd.
- Run config_modules.cmd.
- Run libtbx.scons -j <nproc>.

From bootstrap.py

Below is a cmd script for manually building PHENIX with bootstrap.py. It first downloads sources from external repositories, then compiles them, then runs tests and finally creates a binary installer. It is assumed the script is started with command line arguments specifying platform and version label that will be tagged to the executables during the build. Because the PHENIX build will access repositories that requires authentication you must provide appropriate user names for the flags --sfuser= and --ccuser=.

PhenixBuild.cmd

```
SET PLATFORM=%1
SET PHENIXVERSION=%2

IF DEFINED PLATFORM (
IF %PLATFORM% == x32 (
  call "C:\Program Files (x86)\Microsoft Visual Studio 9.0\VC\bin\vcvars32.bat"
  set PYTHONEXE=C:\Python27_32\python.exe
)

IF %PLATFORM% == x64 (
  call "C:\Program Files (x86)\Microsoft Visual Studio 9.0\VC\bin\vcvars64.bat"
  set PYTHONEXE=C:\Python27_64\python.exe
)

)

REM Get latest bootstrap.py from Github
curl https://raw.githubusercontent.com/cctbx/cctbx_project/master/libtbx/auto_build/bootstrap.py > bootstrap.py
IF ERRORLEVEL 1 (
  EXIT
)

REM hot update base build
%PYTHONEXE% bootstrap.py --builder=phenix --sfuser=oeffner --ccuser=rdo20 --nproc=10 2>&1 | mtee ..\build.log
IF ERRORLEVEL 1 (
  EXIT
)

REM run tests
```

```
%PYTHONEXE% bootstrap.py --builder=phenix --sfuser=oeffner --cciuser=rdo20 --nproc=10 tests 2>&1 | mtee
IF ERRORLEVEL 1 (
    EXIT
)

REM create installer
call build\bin\libtbx.create_installer.bat --binary --version %PHENIXVERSION% --install_script modules\phenix\p
--license modules\phenix\LICENSE.txt --readme modules\phenix\CHANGES --readme modules\phenix\phenix\installer\
--dist_dir dist\%PHENIXVERSION% tmp\phenix-installer-%PHENIXVERSION%-win7vc90 2>&1 | mtee ..\CreateInst
```

In the above script mtee.exe from <https://ritchielawrence.github.io/mtee/> is used for piping stdout to a file as well as to the console. It works similar to tee on Unix platforms. A newer version has been compiled that also retains the error code of the program that streams to stdout. This is useful when checking for the success of the builds. Source code and executable is available from <http://oeffner.net/development/Stuff/mtee.zip>.

Automated build with Buildbot

Assuming the PC runs as a Buildbot slave builds are automated from the Buildbot master machine. Required private keys should be loaded in a logon.cmd script with ssh-agent and command prompts must set the compiler environment variables through an ntshell.cmd script as detailed above.

Issue defining environment variables for the compiler

If you want to run manual builds the environment variables are readily invoked by calling vcvars32.bat or vcvars64.bat. But if doing an automated build with Buildbot all the necessary prerequisites must be accessible in the executable path from a command prompt. At first glance one might add the full path of vcvars32.bat or vcvars64.bat to ntshell.cmd. However, due to a bug in those files this may lead all command prompts to hang in an infinite loop. A workaround is to copy vcvars32.bat and vcvars64.bat to the %HOMEDRIVE%%HOMEPATH% folder and rename them myvcvars32.bat and myvcvars64.bat, respectively. Then replace the lines:

```
:GetWindowsSdkDirHelper
@for /F "tokens=1,2*" %i in ('reg query "%1\SOFTWARE\Microsoft\Microsoft SDKs\Windows" /v "CurrentInstallFolder"
if "%i"=="CurrentInstallFolder" (
    SET "WindowsSdkDir=%k"
)
)
```

with

```
:GetWindowsSdkDirHelper
SET fname=%TEMP%\tmpSDKvars.txt
reg query "%1\SOFTWARE\Microsoft\Microsoft SDKs\Windows" /v "CurrentInstallFolder" &gt; %fname%

@SET WindowsSdkDir=
@for /F "tokens=1,2*" %i in (%fname%) DO (
    if "%i"=="CurrentInstallFolder" (
        SET "WindowsSdkDir=%k"
    )
)
del %fname%
```

then add a line to your ntshell.cmd with the full path of myvcvars32.bat or myvcvars64.bat like:

```
call "C:\Users\builder\myvcvars32.bat"
```

Setting up a PC as a Buildbot slave

The Buildbot slave is installed as per online documentation. For instance for installing a slave named intel-windows-x86_64 in C:\Users\builder\slave chdir to python installed in C:\Python2.7.9x64 type:


```
C:\Python2.7.9x64\Scripts\easy_install.exe buildbot-slave
```

and then:

```
C:\Python2.7.9x64\Scripts\buildslave create-slave C:\Users\builder\slave cci-vm-6.lbl.gov:9989 intel-windows-x8
```

which will make it listen to a master on port 9989 from cci-vm-6.lbl.gov. It is possible to start this slave automatically as a Windows service (equivalent of a Unix daemon) but a simpler procedure is to add the following line:

```
START "Running BuildBot Slave. Do not close!" /MIN cmd /c C:\Python2.7.9x64\Scripts\buildslave start C:\Users\b
```

to your logon.cmd script. Once you have logged on this will place an icon of a minimised command prompt on the taskbar which must not be interfered with, let alone closed. The account can now be locked (Windows key + L) and is ready for doing regular builds from a Buildbot master.

Creating an executable Windows installer program

If the build is done manually from bootstrap.py a Setup.exe program can optionally be created with the last line in the PhenixBuild.cmd script above. If the build is done with Buildbot this installer will currently be created after the various tests have completed. Click here for further details.

Important tweaks for shortening duration of builds

Clear virtual memory at shutdown: Go to Control Panel\All Control Panel Items\Administrative Tools->Local Security Policy->Local Policies->Security Options->Shutdown: Clear virtual memory pagefile

Adjust for best performance of background processes: Check Control Panel\All Control Panel Items\System->Advanced system settings->System Properties | Advanced tab->Performance settings->Performance Options | Advanced tab->Adjust for best performance of Background services

Ensure at least weekly disk defragmentation which does not coincide with doing builds.

If used for nightly builds then disable automatic installation of updates as this often triggers an automatic reboot of the PC. This can be done from the control panel on Windows 7. On Windows 10 follow the steps described on

<http://www.howtogeek.com/224471/how-to-prevent-windows-10-from-automatically-downloading-updates/>

For the directory where nightly builds are done:

Exclude it from Windows indexing. Exclude it from on-access virus scan (both read and write) Avoid storing several previous builds since NTFS doesn't perform well when zillions of files are present on the same disc. Zip up previous builds into single files or move them to a different partition if they have to be retained.

- Exclude it from Windows indexing.
- Exclude it from on-access virus scan (both read and write)
- Avoid storing several previous builds since NTFS doesn't perform well when zillions of files are present on the same disc. Zip up previous builds into single files or move them to a different partition if they have to be retained.

Failure to follow these steps could over time make successive builds very slow (+24 hours)

Using Coot with PHENIX

coot.html

Instructions for use

Coot is an open-source (GPL) model-building program written by Paul Emsley. If a recent version of Coot (0.5 or later) with Python enabled is installed on your system, PHENIX can use it as a viewer for map and model files. The path to the Coot executable will be detected automatically if it is already part of the PATH environment variable or is in one of several standard locations. If not, you will need to enter the path explicitly in the Preferences, as shown below.

The main interface includes a button for launching Coot, and the Utilities menu in most programs contains a "Launch Coot" item. You must start Coot from within PHENIX for the programs to communicate; a single Coot window will be shared between all windows in a single instance of PHENIX. The Graphics pane of the Preferences dialog has an option to automatically start Coot whenever PHENIX is launched.

On Windows it is called WinCoot (courtesy of Bernhard Lohkamp) rather than Coot. Moreover, you would need to specify the full path for the file runwincoot.bat, not just the path of the folder where the runwincoot.bat is located. The file is usually located in the topmost directory where you installed WinCoot.

Once the Coot window appears, you should see additional buttons on the toolbar for toggling communication with PHENIX and showing/hiding hydrogen atoms. There will also be a new menu titled PHENIX. Currently this only enables launching of phenix.refine with an open model pre-loaded. If you do not see any modifications to the Coot window, you probably downloaded a version without Python (see "Common problems" below for details).

As long as Coot is open, any PDB file can be viewed there by right-clicking the magnifying glass icon next to the file name and choosing "Open in Coot" from the menu that appears. If Coot is selected as the default PDB viewer (in Preferences->File Handling), you may simply left-click the icon. Most applications that output models and/or maps will display a button alongside the results that will load the output file(s) in Coot.

In the phenix.refine, AutoBuild, and AutoSol GUIs, the current model and maps will be automatically updated in Coot after every cycle of refinement or building. These models will be colored cyan and are regularly overwritten; use the yellow model if you want to make permanent changes. In phenix.refine, the start model will also be displayed (in purple). To disable these automatic updates, uncheck "Show progress in graphics window" in Preferences->Graphics.

Most applications output either a simple F(obs) map (FP PHI FOM), often after density modification, or a pair of 2mFo-DFc and mFo-DFc difference maps. By default, the F(obs) and 2mFo-DFc maps will be contoured at 1 sigma (1 standard deviation above the mean) and colored blue, and the mFo-DFc map will be contoured at +/- 3 sigma and colored green and red. You can change these colors in the Preferences (Color section).

Common problems

Coot is installed, but nothing happens when I click the button. This probably means that PHENIX couldn't locate it automatically; you will need to enter the exact path in the Preferences (see above). "Coot opens when I click the 'Open in Coot' button, but no model or maps appear." This is almost always due to the Coot installation lacking Python, usually on Linux. If you are using one of the official packages distributed from the Coot web page, you need to download one with "python" in the name. You can double-check this by running "coot --version"; it should mention Python in the output.

- Coot is installed, but nothing happens when I click the button. This probably means that PHENIX couldn't locate it automatically; you will need to enter the exact path in the Preferences (see above).
- "Coot opens when I click the 'Open in Coot' button, but no model or maps appear." This is almost always due to the Coot installation lacking Python, usually on Linux. If you are using one of the official packages distributed from the Coot web page, you need to download one with "python" in the name. You can double-check this by running "coot --version"; it should mention Python in the output.

References

Emsley P, Lohkamp B, Scott WG, Cowtan K. Features and development of Coot. *Acta Cryst.* 2010, D66:486-501. Emsley P, Cowtan K. Coot: model-building tools for molecular graphics. *Acta Cryst.* 2004, D60:2126-32.

- Emsley P, Lohkamp B, Scott WG, Cowtan K. Features and development of Coot. *Acta Cryst.* 2010, D66:486-501.
- Emsley P, Cowtan K. Coot: model-building tools for molecular graphics. *Acta Cryst.* 2004, D60:2126-32.

Dictionary of common terms used in PHENIX

dictionary.html

2mFo-DFc map: one of the most common types of Sigma-A weighted maps generated by modern refinement programs, and the primary map used for manually inspecting and building the model in between rounds of refinement. Fo and Fc are the experimentally measured and model-based amplitudes, respectively (Fc here is synonymous with F(model)). m is the figure of merit, and D is the Sigma-A weighting factor. The effect of this combination of factors will be to amplify the regions of the map in which portions of the true model are missing, and to better account for errors in the model-based amplitudes. This is technically a type of difference map, although the term is often used specifically to refer to mFo-DFc maps and similar. Usually viewed at between 1.0 to 1.5 sigma (the latter is the default in Coot).

Alternate conformations: at high resolution (usually significantly better than 2.0Å) the map may be sufficiently detailed to recognize two or more unique conformations for parts of a molecule. The affected residues (here meaning any connected set of atoms, not necessarily amino acids) can be split into multiple conformers with partial occupancy summing to 1. At present these need to be built manually in Coot.

altloc: an identifier in atom records in PDB and mmCIF files, which labels the alternative conformation to which an atom belongs. In the PDB format, this occupies the column to the left of the residue name. For single-conformer models the altloc ID will be blank; otherwise it is usually an uppercase letter starting from "A". Atoms with non-blank altloc IDs will interact with unlabeled atoms and those with the same identifier, but not atoms with a different non-blank altloc.

Anisotropic: not equal in all directions, or in practice, ellipsoidal in shape. In crystallography this is used in two very different contexts:

Anisotropic B-factors model the atomic displacement as an ellipsoid, which introduces six parameters per atom (versus one for isotropic B-factors). This is more accurate, but refinement of individual B-factors using this parameterization is only possible at atomic resolution (approximately 1.5Å or better, depending on data quality and solvent content). At moderate resolution, TLS parameters for groups of atoms can be refined instead. Anisotropic data does not extend to the same resolution in all directions; typically, the anisotropy is along the direction of the unit cell axes (or h,k,l in reciprocal space). This is an effect of varying crystal quality. Programs such as phenix.refine and Phaser will perform anisotropic scaling during phasing, refinement, and map generation to compensate for some of the effects of anisotropy, but in some cases the diffraction images will need to be processed specially.

- Anisotropic B-factors model the atomic displacement as an ellipsoid, which introduces six parameters per atom (versus one for isotropic B-factors). This is more accurate, but refinement of individual B-factors using this parameterization is only possible at atomic resolution (approximately 1.5Å or better, depending on data quality and solvent content). At moderate resolution, TLS parameters for groups of atoms can be refined instead.
- Anisotropic data does not extend to the same resolution in all directions; typically, the anisotropy is along the direction of the unit cell axes (or h,k,l in reciprocal space). This is an effect of varying crystal quality. Programs such as phenix.refine and Phaser will perform anisotropic scaling during phasing, refinement, and map generation to compensate for some of the effects of anisotropy, but in some cases the diffraction images will need to be processed specially.

Visualization of individually refined isotropic (left) versus anisotropic (right) B-factors for a high-resolution structure

Anomalous scattering: a phenomenon in which the scattering of X-rays by electrons has both imaginary and real components, leading to breaking of Friedel's law. Anomalous scattering is most pronounced in heavy atoms and is wavelength-dependent; each atom has several "edges" at which the effect reaches a maximum. The change in scattering at a given wavelength is represented by two numbers, f' (f -prime), which describes the "dispersive difference" or change in the real component, and f'' (f -double-prime), which describes the "anomalous difference" or change in the imaginary component. Anomalous scattering has several uses:

The result will be small but measurable differences in Friedel pairs (F^+ and F^-), which can be exploited to locate the heavy-atom substructure and phase the entire structure, as used in SAD and MAD phasing methods. Anomalous data may also be used after phases are available to calculate an anomalous difference map, which also shows the location of heavy atoms. Because this map is automatically calculated in phenix.refine and is very useful in completing the model, we recommend refining against anomalous data if available, even if no additional anomalous scatterers were introduced as part of the experiment. In addition to map calculation, it is also possible in phenix.refine to refine the anomalous scattering of specific atoms, which may be more accurate than using the non-anomalous scattering factors.

- The result will be small but measurable differences in Friedel pairs (F^+ and F^-), which can be exploited to locate the heavy-atom substructure and phase the entire structure, as used in SAD and MAD phasing methods.
- Anomalous data may also be used after phases are available to calculate an anomalous difference map, which also shows the location of heavy atoms. Because this map is automatically calculated in phenix.refine and is very useful in completing the model, we recommend refining against anomalous data if available, even if no additional anomalous scatterers were introduced as part of the experiment.
- In addition to map calculation, it is also possible in phenix.refine to refine the anomalous scattering of specific atoms, which may be more accurate than using the non-anomalous scattering factors.

Example of an anomalous difference map, calculated from data collected at the selenium K edge (approximately 0.9792 Å wavelength) on a selenomethionine-derivatized protein crystal. The anomalous map is rendered in purple, contoured at 3.0 sigma, and clearly shows the positions of the Se atoms in the structure.

ANISOU: a type of record in PDB format, specifying an anisotropic displacement ellipsoid with six parameters. They will immediately follow the associated ATOM or HETATM record. ANISOU records represent the upper right corner of the symmetric matrix of atomic displacement. ANISOU records will appear in files output phenix.refine any time you use individual anisotropic B-factor refinement or TLS. Anisotropic B-factor is related to isotropic B-factor reported in ATOM record: isotropic B is the mean of the trace of ANISOU matrix divided by 10000 and multiplied by $8\pi^2$ and represents the isotropic equivalent of the total B-factor.

B-factors or Atomic Displacement Parameters (ADPs): B-factors or ADPs describe the variability or uncertainty in atomic positions, and (unlike occupancy) have a resolution-dependent effect on the diffraction amplitude. The simplest model for these (isotropic) is a sphere centered around the atomic position, but at high resolution (approximately 1.5 Å or better) it becomes possible to model them as ellipsoids (anisotropic). At low resolution (worse than 3-3.5 Å) it may be necessary to refine ADPs for small groups such as entire residues, instead of individual atoms. The total ADPs will also include contribution from larger displacements such as those modeled by TLS refinement and an overall anisotropic ADP for the entire crystal. (Note that in other contexts, "ADP" may instead mean "anisotropic displacement parameter".)

B-factor sharpening: a method to improve map quality (or interpretability) by systematically scaling reflections in a resolution-dependent manner. This may be applied either to the map coefficients directly, or the underlying experimental data. (This is different from the B-factors or ADPs of the actual atoms, but the same term is used because it represents the intensity falloff as a function of resolution.) "Sharpening" actually means to increase the high-resolution amplitudes, but these can also be scaled down, in which case the term

"blurring" is more accurate. The map calculation methods in Phenix have an option to apply B-factor sharpening (and it may be done automatically in some contexts), but this can also be done interactively in Coot.

Asymmetric unit: smallest fraction of the unit cell such that applying all crystallographic symmetry operations to it will generate the entire unit cell.

Bijvoet mates: F^+ and F^- , which are related by Friedel's law, except to the extent that anomalous scattering is present. Also sometimes called "Friedel pairs".

Bulk-solvent correction: Modeling scattering contribution arising from disordered (bulk) solvent. Best available model for bulk-solvent is so-called Flat model, which assumes bulk-solvent electron density to be constant (flat) outside of macro-molecular region. Because the bulk solvent comprises such a large scattering mass in macromolecular crystals (between approx. 20 and 80%), it is essential to account for it. Bulk-solvent mostly contributes to low resolution reflections (approx. lower than 4Å) and its contribution to higher resolution reflections is almost zero. Bulk-solvent is accounted for by adding extra contribution (F_{bulk}) to atomic model structure factors (F_{calc}) to form the total model structure factor $F_{\text{model}} = \text{scale} * (F_{\text{calc}} + F_{\text{bulk}})$. Not accounting for bulk-solvent may significantly impact R-factors, refinement and maps.

C-beta deviations: a type of validation outlier specific to amino acid geometry. If the backbone and/or sidechain are misfit, the C-beta atom may be moved far from the ideal position to compensate. A deviation of more than 0.25Å is considered to be a validation outlier. This will not necessarily correspond to chirality or bond angle deviations, but it is considered a significant error and should never be present in the final model.

Clashscore: a validation statistic used in the Molprobtity web server and the related validation tools in Phenix (on the command line, the program phenix.clashscore can also be used). It is equal to the number of severe atomic clashes (overlaps greater than 0.4Å) per 1000 atoms. A well-refined structure should have a clashscore below 20; clashscores of zero are exceptionally uncommon and very difficult to obtain. Note that while the VDW restraints used in refinement will help prevent the clashscore from increasing, many severe clashes will require manual rebuilding to fix. Clashes can be visualized in KING or Coot as "Probe dots"; these will be generated by Phenix as part of validation.

Example of Probe dots for a severely misfit sidechain. The severe clashes (many caused by overlapping hydrogen atoms, not shown) are displayed as pink lines; these count towards the total clashscore.

Completeness (of data): number of measured reflections expressed as a fraction of the total number of reflections theoretically possible in the resolution range of measured data (d_{min} , d_{max}). Sometimes resolution range to count theoretically possible reflections may be (d_{min} , infinity).

Composite omit map: See omit map.

Constraints: In refinement, constraints force specific parameters to be refined together. Examples of this include rigid-body refinement (atomic coordinates constrained), "riding" hydrogen atoms, or grouped ADPs. Since constraints reduce the number of refined parameters, they are particularly useful at low resolution. See also restraints.

Crystallographic Information Format (CIF): a type of structured file format developed specifically for crystallography. Although CIF files may be used to represent any type of data, they are most commonly used in Phenix to define geometry restraints, which are generated primarily by eLBOW. They are also used by the PDB to store reflection or coordinate data, the latter as a substitute for the (archaic) PDB format. Phenix can either convert these files to a more familiar format or use them directly as input in many cases, and phenix.refine can also output both data and model in CIF format. See the overview of file formats for more information.

Density modification (DM): An iterative procedure of applying a priori knowledge about fundamental properties of crystal model (atomic model and bulk-solvent) and density map in order to improve structure factor phases and in some cases amplitudes. Such a priori knowledge includes: atomicity, positivity, noncrystallographic symmetry, bulk-solvent solvent flatness, map connectivity, expected map histogram, etc. Two approaches to DM exist: classical and statistical. DM may dramatically improve maps. For SAD phasing, density modification is essential to resolve phase ambiguity and obtain an interpretable map, but it is also very helpful for other methods.

Above: an example of an experimentally phased SAD map (the p9-sad tutorial distributed with Phenix) before and after density modification.

Difference map: any map calculated from the difference in amplitudes between two sets of structure factors (with phases from one of these, or from an external source). This can include an anomalous difference map, which shows the location of anomalously scattering atoms, an isomorphous difference map, which shows atoms present in one crystal but not another, or any of several types of maps which show the difference between the experimental data and the model, of which the mFo-DFc map is the most common. (When used without qualifications, the term "difference map" will often simply refer to the mFo-DFc map.)

Direct methods: a phasing method based on mathematical relationships between reflections, which does not require any additional information beyond the diffracted amplitudes. Direct methods are standard for small-molecule crystallography, but because they require atomic-resolution data and only work for relatively small structures (typically 1000 atoms or less), they are rarely able to phase protein crystal data. However, they can be used to locate heavy atom sites if anomalous data of sufficient quality are available; in Phenix, the program HySS uses direct methods to perform the heavy-atom search. Herbert Hauptman and Jerome Karle won the Nobel Prize in Chemistry in 1985 for their work related to direct methods in reciprocal space. However, direct methods are more intuitively understood in the direct-space formulation, based on the Sayre equation.

Direct summation: a simple and accurate (but computationally inefficient) method for converting between reciprocal-space and real-space data, by summing over all reflections and all atoms (or grid points in the unit cell). It is still widely used in small molecule crystallography, but programs such as Phenix use the FFT method which is far more efficient.

d_min: High-resolution limit.

d_max: Low-resolution limit.

Experimental phasing: umbrella term for a variety of phasing methods based on exploiting the scattering of heavy atoms (the heavier the better, but even endogenous atoms such as sulfur may be used in favorable cases). Includes SAD, MAD, and isomorphous replacement. The majority of new structures are determined by molecular replacement, which is usually significantly easier if a similar structure has already been solved, but experimental phasing is usually the only option for genuinely novel structures, and has the added advantage of avoiding model bias in the resulting phases, especially at lower resolution.

f-prime (f') and f-double-prime (f''): see anomalous scattering.

Fast Fourier Transform (FFT): a computationally efficient method for converting between reciprocal space and real-space data (that is, for converting amplitudes and phases to electron density and vice-versa). It is typically orders of magnitude faster than direct summation. In addition to being used whenever a map is displayed, the FFT is at the core of many algorithms in Phenix (and macromolecular crystallography in general), including the calculation of structure factor target and gradients in phenix.refine.

Figure of Merit (FOM): An approximate measure of phase quality, calculated for each reflection. Range is from 0 to 1, higher being better. The FOM is defined as the expected value (probability-weighted average) of the

cosine of the phase error; Blow & Crick showed that if this weight is applied to terms in an electron density calculation, the RMS error in the density is minimized. Although it is sometimes reported in the context of refinement and density modification, it is only useful for evaluating initial experimental phases. A good MAD solution will typically have an FOM greater than 0.4, while a SAD solution should usually be greater than 0.3.

F(model): the structure factors calculated from the model, which in this context also includes the bulk-solvent contribution. Sometimes also referred to as "F(calc)".

F(obs): experimentally observed amplitudes.

Friedel's law: a phenomenon in which pairs of reflections related by a center of inversion, (h,k,l) and (-h,-k,-l), have identical amplitudes (but conjugate phases). Anomalous scattering breaks the Friedel symmetry, which can be taken advantage of to locate heavy-atom sites and phase the structure.

Gradient: the first derivative of a function (such as the experimental data or geometry restraints targets), used for minimization.

Grid search: an optimization method that samples a series of values for the refined parameters and picks the best result, as opposed to using gradient minimization or simulated annealing. Grid searches tend to be much slower but the ability to try any arbitrary value gives them a much larger radius of convergence than other methods, and they are especially useful when a pre-defined set of trial values is known. In Phenix, grid searches are used for sidechain rotamer fitting, selection of the relative refinement target weights, and optimization of bulk solvent parameters.

Hendrickson-Lattman coefficients: Phase probability parameters, calculated either experimentally based on heavy-atom methods (SAD, MAD, etc.) or directly from the model, used in density modification and (if experimentally obtained) as additional restraints during refinement. Consists of four coefficients per reflection, which describe a bimodal phase distribution. (If calculated from a model, the distribution will in fact be unimodal, but the same representation is used by convention.)

Insertion code: a single column in the ATOM records of a PDB file, used to supplement the residue number. Insertion codes are used when a specific numbering convention is desired, e.g. to preserve the identity of key residues in related proteins, or to reflect an expressed sequence modified from the genomic equivalent. In these cases, consecutive residues might be numbered 25, 26, 26A, 26B, 26C, 27, etc. When defining atom selections in Phenix, the syntax "icode A" is used; the insertion code can also be combined with the residue number to uniquely identify a residue, using the syntax "resid 26A".

Intensities: the measurements obtained from the diffraction experiment, i.e. the values of spots on the detector. The intensity of a given reflection is proportional to the square of the amplitude, times a scale factor that is determined by photon flux, crystal size, and other experimental properties. Note that because intensities are calculated by subtracting the spot from the background, they may be negative (unlike the amplitude); this is corrected by French-Wilson treatment instead of simply taking the square root of positive intensities and discarding the rest. Because this conversion is performed internally, many of the programs in Phenix accept either intensities or amplitudes as input, although internally the latter are nearly always used.

Interatomic scattering: X-ray scattering caused by the electrons in the covalent bonds between atoms. At very high resolution ($d_{\min} < 0.9\text{\AA}$), this is sufficiently distinct from the atomic scattering that it may appear as difference map peaks. In phenix.refine, this can be modeled by additional interatomic scatterers, essentially pseudo-atoms. See Afonine et al. 2007 for more information.

Above: an example of unmodeled interatomic scattering (visible as green mesh representing positive peaks in the mFo-DFc map) in the 0.66Å structure of aldose reductase (PDB ID 1us0
<<http://www.rcsb.org/pdb/explore/explore.do?structureId=1us0>>`__).

Isomorphous: geometrically similar, which in the context of crystallography means that the space group is the same and the unit cell parameters are nearly the same between two crystals/datasets. The degree to which the unit cell parameters are allowed to deviate before datasets are considered non-isomorphous may depend on the specific method being used to compare or combine the datasets, but the maximum permissible change in unit cell length is usually 1% (other rules based on resolution are also sometimes used). By default many programs in Phenix use very strict rules for determining isomorphism, although some of these can be modified or ignored.

Isomorphous replacement: an older method for phasing protein crystal structures, in which a native dataset is collected, followed by one or more heavy-atom derivatives that are isomorphous to the native data, and the differences in amplitudes are used to locate the heavy-atom substructure and calculate experimental phases. For many years isomorphous replacement was the standard tool for phasing, but it is very time-consuming compared to SAD or MAD, let alone molecular replacement, and is rarely used now except for very difficult structures. Several variants exist: SIR (single isomorphous replacement), which like SAD requires density modification to resolve the phase ambiguity; MIR (multiple isomorphous replacement); and SIRAS or MIRAS, in which the anomalous signal of the heavy atoms is also employed. In Phenix, isomorphous replacement is performed by the AutoSol wizard.

Isotropic: moving equally in all directions. In crystallography, this is usually used to describe the simplest (and most common) parameterization of B-factors or atomic displacement parameters (ADPs). Only one parameter per atom is refined, the mean atomic displacement u (where $B = 8 * \pi^2 * u^2$). Visually, the B-factor is represented as a sphere. See also anisotropic and TLS.

Kicked map: A faster alternative to omit maps. All coordinates are shaken randomly by a small amount, and the map recalculated; this is done repeatedly and the maps averaged. This can help remove bias and in some cases noise as well. Available in both phenix.refine and phenix.maps. See Praznikar et al. 2009 for details.

Least squares: an older refinement methodology, no longer in widespread use for macromolecular crystallography. A least-squares target assumes that the error in experimental observations takes a Gaussian distribution; modern maximum-likelihood targets do a better job weighting the observations and accounting for sources of error. (The optimization methods used, such as minimization or simulated annealing, are the same regardless of the specific target.) In Phenix, a least-squares target is still used when performing twinned refinement, or refinement against a very small number of reflections.

Log-likelihood gain (LLG): a statistical measure used by Phaser; in the context of molecular replacement, it essentially scores the model placement in comparison to a random model. The LLG should be positive and relatively large if the MR solution is correct.

LLG map: as the name indicates, this is a map calculated using the gradient of the log-likelihood as "structure factors". In the context of SAD phasing, the LLG map shows the difference in anomalous scattering between the current substructure and the true contents of the crystal. This has the effect of amplifying the map around weaker, unmodeled scatterers. In Phaser the process of substructure completion is automated and iterative, but it is also possible to output an LLG map from phenix.refine and phenix.maps. If no anomalous scattering has been modeled, the LLG map will look very similar the conventional anomalous difference map.

Map coefficients: This is simply the Fourier coefficients for an electron density map, in MTZ format. Graphics programs such as Coot (and PHENIX, internally) will perform the Fourier transform automatically to obtain the real-space density. Most programs in PHENIX output map coefficients rather than pre-calculated maps, but the terms are often used interchangeably.

Maximum entropy: A statistical technique for noise removal; in the context of crystallography, it is used to reduce artifacts in electron density maps caused by truncation of Fourier series due to limited resolution or missing data. Not widely used in macromolecular crystallography, but it is helpful in some cases. An

implementation is available in Phenix.

Maximum likelihood: The statistical methodology used in target functions for refinement and phasing, in which the probability of the model given the data is maximized. Maximum likelihood is much better at weighting the data appropriately than the previously used least-squares method, and leads to greater sensitivity and improved maps. See McCoy *Acta Cryst.* (2004). D60, 2169-2183 for an introduction to the method as used in crystallography.

mmCIF: see CIF.

mFo-DFc map: the other most common type of map generated by refinement programs, along with 2mFo-DFc. Colloquially called a "difference map", although technically 2mFo-DFc maps also fall into this category. The mFo-DFc map is usually viewed at positive and negative contours (typically ± 3 sigma). The positive density indicates features present in the data that are not accounted for by the model; the negative density indicates parts of the model that are not supported by the data. Note that in a well-refined structure some residual difference density is always expected, and interpretation of the maps needs to also take into account the 2mFo-DFc map and local model features.

An example of 2mFo-DFc and mFo-DFc difference maps. The 2mFo-DFc map is colored blue and contoured at 1.0 sigma (i.e. 1.0 standard deviations above the mean electron density), and shows where we expect most of the model to be (excluding hydrogen atoms, not included here). The mFo-DFc map is colored green (3.0 sigma) and red (-3.0 sigma); the green mesh shows where atoms are missing in the current model, while the red mesh shows where atoms are present in the model but not the crystal. In this image, the central Tyr rotamer is clearly misfit, as is the carboxyl group of the Glu sidechain in the upper left corner.

Miller array: Any set of reciprocal-space data indexed by h,k,l ("Miller indices"). This can include experimental intensities or amplitudes, phases, weights (such as FOM), Hendrickson-Lattman coefficients, R-free flags, etc. These are the basic unit of reflection data in PHENIX, and may combine multiple columns from the input file (for instance, F and SIGF in an MTZ file will be grouped into a single Miller array containing both amplitudes and sigmas).

MIR: see Isomorphous replacement.

Model bias: a term used to describe the phenomenon in which the phases calculated based on the model will tend to result in an electron density map that resembles the model, regardless of the amplitudes used. Several excellent 2D examples of this can be viewed on Kevin Cowtan's Book of Fourier. In macromolecular crystallography, model bias is often a serious concern due to the limited resolution of most datasets, and maps calculated using model phases need to be interpreted with a healthy amount of skepticism. Methods for minimizing model bias include the calculation of omit maps or the use of experimental phases. At atomic resolution model bias is less problematic, but for low-resolution structures (especially below 3.0Å) the effect is both pervasive and difficult to avoid.

Molecular replacement (MR): the most popular phasing method, accounting for more than three-quarters of all new PDB depositions. MR uses known structures to phase (and also provides an initial model for refinement), by determining their rotation and translation in the unit cell of the collected data. Unlike other phasing methods (SAD, MAD, MIR, etc.), MR does not require a special experimental setup or especially high-quality data. The search model may either be a single PDB file, an ensemble of superimposed PDB files of similar structure, or a processed electron density map (containing only the region of interest). The search model is often nearly identical to the target structure (e.g. when the goal is a known protein in complex with a ligand, or a point mutant), but search models with sequence identities as low as 30-40% are usually still easily solved, and significantly lower-identity models may still be used with additional processing (particularly trimming non-conserved loops and the ends of non-identical side chains in Sculptor) if their structures are similar enough. In Phenix, MR is performed by Phaser and various procedures that invoke Phaser.

Molprobity score: an attempt to provide an overall measure of protein structure quality based on the distribution of validation statistics in structures deposited in the Protein Data Bank. The score is defined in (Keedy et al. 2009) as:

$$\text{MPscore} = 0.426 * \ln(1 + \text{clashscore}) + 0.33 * \ln(1 + \max(0, \text{rota_out} - 1)) + 0.25 * \ln(1 + \max(0, \text{rama_iffy} - 2)) + 0.5$$

with clashscore as defined above, rota_out is the percentage of sidechains with rotamer outliers, and rama_iffy is the percentage of amino acids not in the "favored" region of the Ramachandran plot. The resulting score is on the same scale as resolution in Angstroms, so a structure with a MolProbity score of 2.4 is similar in quality to structures in the PDB at 2.4Å. The score should usually be smaller than the actual resolution, the lower the better.

MTZ: a binary format for reflection data, developed as part of the CCP4 suite. It can contain essentially any combination of data commonly used in refinement, including amplitudes or intensities (with or without sigmas), phases, Hendrickson-Lattman coefficients, and various integer arrays (including R-free flags). Each one-dimensional data array in an MTZ file has a unique "column label", although Phenix will often work with groups of these arrays, for instance grouping "F" and "SIGF" together if they appear sequentially in the file. MTZ files are usually the preferred format for reflections files, as they are very versatile, reasonably compact, and supported by a wide range of modern software, including Phenix, CCP4, and Coot. See the overview of file formats for more information.

Multi-wavelength Anomalous Diffraction (MAD): a popular method for experimental phasing using heavy atoms. MAD takes advantage of the tunability of synchrotron beamlines to collect data for (ideally) a single crystal at multiple wavelengths (at least two, but rarely more than four), clustered around the anomalous "edge" of the heavy atom of interest (most commonly selenium, but any element whose edge is accessible with synchrotron radiation is suitable, including many heavy metals). The differences in anomalous scattering around the edge allow calculation of phase angles without the phase ambiguity present in SAD experiments, although density modification will usually still be necessary to obtain an easily interpretable map. Because of the sensitivity to small differences in f' and f'' , which diverge significantly from theoretical values around the anomalous edges, these values should be experimentally measured at the beamline (or suitable approximations obtained from beamline staff). Although very powerful, MAD phasing has declined somewhat in popularity relative to SAD because of the more limited choice of heavy atoms, the difficulty of avoiding radiation damage, and the requirement for a synchrotron beamline, but it is still a much easier method than isomorphous replacement. In Phenix, MAD is performed by the AutoSol wizard, using the underlying program SOLVE.

Non-crystallographic symmetry (NCS): Symmetry within a crystal that is independent of the lattice type; this is very common in biomolecules, especially homo-oligomeric proteins. Information about NCS can be used to great advantage during density modification, and as additional restraints on the conformation of related groups during refinement. (PHENIX does not currently support NCS constraints, which force related molecules to be identical.) NCS is most easily identified by comparing chains in a model, but can also be detected by analysis of heavy-atom sites or even electron density; several tools exist in Phenix for this purpose, and it can be performed automatically by the AutoSol wizard and phenix.refine. See the refinement FAQs for additional information about the use of NCS restraints.

Occupancy: an attribute of atoms in a structure, equal to the fraction of unit cells in which the atom occurs in the given position. High-resolution structures (usually 1.6Å or better) will often need occupancy refinement for sidechains that occur in more than one conformation, or for ligands that are not 100% bound. The occupancy should never be set to zero in a deposited structure. Note that B-factor and occupancy are correlated in practice and the effects of each may be difficult to tell apart; however, partial occupancy has an equal effect at any resolution, unlike the B-factor whose effect is resolution-dependent.

Omit map: a map generated by deleting part of the structure (for instance, a bound ligand) and recalculating phases and $F(\text{model})$. This almost always some procedure to remove phase bias, either by iterative re-phasing from a modified $F(\text{model})$, refinement of the modified structure (especially with simulated annealing), or rebuilding. A variant, the composite omit map, stitches together the contribution of many individual omit maps, which collectively omit all atoms. Phenix has a standalone composite omit map program that performs most of these procedures; the iterative-build omit map can be calculated using the AutoBuild wizard. A more detailed explanation of options is available in the frequently asked questions list. The removal of phase bias is discussed in Hodel et al. 1992 and Pražnikar et al. 2009.

Overfitting: Optimization of R-work at the expense of other quality metrics, especially R-free. The most obvious indication of overfitting is divergence of R-work and R-free, which ideally should decrease in sync throughout refinement. In most cases, the solution is to add or tighten restraints, or reduce the number of refined parameters.

Phasing: determination of the missing phase angles to accompany the experimentally measured amplitudes. Obtaining the phases allows calculation of the Fourier transform of the reflection data to obtain the electron density into which a model can be built. Several methods for reconstructing the phases are possible (covered elsewhere in this document): Molecular replacement, experimental phasing (SAD, MAD, or isomorphous replacement), and direct methods (which is not generally useful for most macromolecular structures). For crystals that are essentially isomorphous to a known structure, the previously determined phases (often in the form of a model) can also be used directly.

Radius of convergence: a term used to describe how far a model can be improved from a given starting point. The larger the radius of convergence, the worse a starting model can be without stalling refinement, and the better the result. The radius of convergence is affected by multiple refinement options, including the choice of strategy (for instance, rigid-body refinement has a very large radius of convergence for crude models), the optimization target, and the optimization method. Simple gradient minimization usually has a smaller radius of convergence than simulated annealing and grid search methods (such as rotamer fitting), but it is usually significantly faster to run. When a structure is described as being "beyond the radius of convergence" of a program, this means that it cannot be improved by automatic methods. Programs such as MR-Rosetta are designed to have a very wide radius of convergence, although at the cost of long run times.

Ramachandran plot: a two-dimensional graph of the ϕ, ψ angle combinations of their allowed backbone; also refers to the expected/allowed distribution of points on this graph. This distribution is dictated by steric constraints on the backbone conformation, originally identified by Ramachandran et al. (1963) *J Mol Biol* 7:95-99. Because the Ramachandran plot is an essential validation metric, ϕ and ψ are typically left unrestrained during refinement. In the Molprobity server and the Phenix validation tools, the plot is divided into "favored", "allowed", and "outlier" regions, based on the distribution of angles in a set of 8000 high-quality structures; a well-refined structure should have 98% of residues favored, and less than 0.2% outliers, although at lower resolutions it may be difficult to obtain these statistics. Note that the expected distribution varies depending on residue type and environment; in the current version of Phenix, six different distributions are used.

A representative Ramachandran plot, with outliers shown in red; the contours define the borders of the favored and allowed regions.

Real space: coordinates in the crystal (relative to an origin which may be somewhat arbitrary depending on the space group). Examples of real-space data are model coordinates and electron density maps.

Real-space refinement: coordinate refinement using an electron density map (usually the $2mFo-DFc$ map) as the experimental target. This may be done globally or locally, and with any optimization method, including gradient minimization or grid search (such as rotamer fitting). It is much faster than reciprocal-space

refinement, but has the disadvantage of depending on phases being accurately calculated. To avoid biasing R-free, the reflections flagged as the test set should be left out of the map calculation. (This is done automatically when run as part of phenix.refine.)

Reciprocal space: a mathematical construction describing the positions of reflections in a "reciprocal lattice", whose parameters are directly related to the crystal lattice parameters; the reciprocal lattice vectors are perpendicular to real-space planes. The dimensions in reciprocal space are expressed in terms of 1/Angstrom, i.e. the reciprocal of the resolution for a given reflection, and positions are indexed by the Miller indices h, k, l. The process of data collection can be thought of as sampling reciprocal space, which rotates along with the crystal lattice.

Reflection: an individual data item in reciprocal space, usually used to describe a single amplitude or intensity.

Residual map: see Difference map.

Resolution factor: this number controls the spacing of the grid used for map calculations, which is approximately defined by $d_{\min} \times \text{resolution_factor}$ (although the final grid dimensions will be adjusted based on the requirements of the FFT algorithm). The lower the resolution factor, the more finely spaced the map grid. A spacing of 1/3 or 1/4 is most common, with the latter preferred for display purposes.

Restraints: In refinement, restraints keep specific independent parameters from diverging too far. At most resolutions, because of the limited resolution of the data basic geometry restraints (bonds, bond angles, dihedral angles, chiral centers, planar groups, and VDW interactions) and ADP similarity restraints will be used, usually taking the form of a simple harmonic potential, mimicking a spring pulling parameters back to ideal values. Depending on the specific structure and data, additional restraints may be used, for instance between related molecules (NCS or a reference structure). Restraints do not reduce the number of refined parameters, but are essential to maintain proper geometry and prevent overfitting. However, they become less necessary as resolution increases, and restraining a high-resolution model too tightly will actually make it worse. See also constraints.

Above: an example of refinement with and without geometry restraints at moderate resolution (1.8Å); the molecular geometry is severely distorted if dictated by the experimental data alone.

Above: the same example with and without ADP restraints (shown as spheres, and colored by isotropic B-factor), showing unphysical distribution of B-factors when restraints are not used.

R-factor: Crystallographic residual; for calculating model-data agreement, it is equal to $\text{sum}(\text{abs}(\text{abs}(\text{Fobs}) - \text{abs}(\text{Fcalc}))) / \text{sum}(\text{abs}(\text{Fobs}))$. R-merge, R-sym, etc. are calculated similarly for intensities, over multiple redundant observations instead of calculated values. Generating Fc at random will typically result in an R-factor of 0.55 for an untwinned structure, which is therefore the threshold for determining if a model is placed correctly. (Note that in practice, molecular replacement solutions often have higher starting R-factors, but these will rapidly drop during refinement if the solution is correct. Also note that in the presence of twinning, a lower threshold for randomness is expected.)

R-free: R-factor calculated from reflections not used in refinement, typically 5-10% of the data or 1000-2000 reflections (in statistical terminology this is called "cross-validation"). If the refinement was parameterized appropriately, the R-free should be reduced along with the R-factor for the "working" set (R-work), although it will always be a higher value. An increase in R-free indicates an incorrect optimization, even if R-work decreases. A large spread between R-work and R-free is a primary symptom of overfitting. See Brunger AT (1992) Nature 355:472-475 for full rationale.

R-free flags: an integer array that marks each reflection for use in either refinement or the calculation of R-free. Usually a set of R-free flags is generated at the beginning of the model-building and refinement process, and used throughout (usually as part of a single MTZ file including experimental data); programs

such as AutoSol, AutoBuild, and phenix.refine will do this automatically if no flags have previously been generated. Extension of an existing set of flags to higher resolution, or transfer to an isomorphous dataset, can be done by the reflection file editor.

Riding hydrogens: hydrogens whose coordinates and B-factors are defined by the covalently bonded heavy atom, as opposed to individual refinement. Except for sub-atomic resolution structures, the riding model is almost always the appropriate one, and leads to less overfitting.

Rigid-body refinement: coordinate refinement using rigid blocks of atoms, typically an entire domain or chain (or possibly the entire contents of the asymmetric unit). Used at the beginning of refinement after a molecular replacement search, where the fit to data may be crude and a large radius of convergence is desired. (Note that some rigid-body refinement is also performed directly by Phaser.)

Rotamer: A well-defined unique combination of dihedral angle values in a group of atoms; the term is almost always used in reference to amino acid sidechains, although RNA backbone angles are also rotameric. The sidechain rotamers used in Phenix are extracted from a set of high-quality, high-resolution structures (the Top500 database). The vast majority of sidechains in a finished structure should be recognizably rotameric unless the density very clearly supports an outlier conformation, and flagging of rotamer outliers is an important part of validation. Because the standard geometry restraints and minimization methods often do a poor job moving sidechains into rotameric positions, a separate rotamer-fitting step is available in phenix.refine.

Above: the sidechain rotamers of lysine defined by MolProbity, based on conformations found in the Top500 database.

Rotation function (RF): the first part of molecular replacement, which establishes the rotational orientation of the search model in the crystal. This is independent of the exact space group in some cases, e.g. the same rotation will be correct whether the space group is P6(2) or P6(4). The resulting Z-score is only a weak indicator of whether the rotation is correct or not, i.e. a high Z-score does indicate a correct rotation, but a rotation with a low Z-score may well be correct too.

Scattering factors: values describing the scattering of X-rays by a given atom type, related to the number of electrons in the atom or ion. These are used in refinement to calculate the expected electron density and structure factors given a set of atomic parameters (XYZ coordinates, B-factor, occupancy, and possibly anomalous scattering). A variety of parameterizations of scattering factors are available in Phenix, but the most commonly used (for X-ray refinement) is called "n_gaussian". Scattering factors for neutron diffraction experiments (where the nucleus, rather than the electrons, is responsible for the scattering) are also available.

Sharpening: see B-factor sharpening.

Sigma: a common shorthand term for standard deviation. May occur in several contexts in macromolecular crystallography, most commonly referring to the sigma-scaling of maps.

Sigma-A weighting: a method of weighting difference map coefficients to more accurately account for errors in the model. This is standard practice in macromolecular crystallography, as it leads to improved maps. A sigma-A weighted map will typically be described as "2mFo-DFc" or "mFo-DFc", with the "D" being the relevant value ("m" is the Figure of Merit for each reflection). The maximum likelihood target in Phaser is a derivation of this methodology. See Read, R.J.: Acta Cryst. A42 (1986) 140-149 for original derivation.

Sigma scaling: a method of scaling electron density maps in real space, in which the statistics for the entire unit cell are calculated, and the grid point values are set to the number of standard deviations (sigmas) from the mean value. Although this can be thought of as crude normalization, and the maps are commonly viewed at standard sigma levels (e.g. the Coot defaults of 1.5 sigma for 2mFo-DFc and +/- 3.0 sigma for mFo-DFc maps), the sigma values are actually very subjective, depending greatly on solvent content and model quality.

Simulated annealing: An optimization method which simulates heating up a system and slowly cooling it down, as a way of escaping local energy minima trapping simple gradient-based minimization methods. In crystallography, this means running a simple molecular dynamics simulation starting at very high temperatures (2500-5000 K), with the agreement with X-ray data included as an additional pseudo-energy term. Especially useful for poorly built structures early in refinement, and as a method to remove phase bias (e.g. for omit maps). In PHENIX, both Cartesian and torsion-angle dynamics are available; the latter is intended for low-resolution refinement as it uses fewer parameters. However, in practice Cartesian simulated annealing in phenix.refine often outperforms torsion-angle dynamics. Originally described in Brunger (1997).

Single-wavelength anomalous diffraction (SAD): probably the most popular experimental phasing method currently in use, SAD requires only a single wavelength from one crystal. The experimental phases themselves will be of poor quality because the SAD experiment cannot resolve the phase ambiguity, but density modification will lead to a significantly improved map in most cases. SAD is often performed with selenomethionine-incorporated protein, but any anomalously scattering atom (including sulfur, if the data are of very high quality) may be used. In Phenix, SAD may be performed by either the AutoSol wizard or Phaser-EP, although the latter only calculates initial phases and completes the heavy-atom substructure, whereas AutoSol will also perform initial heavy-atom location, density modification, and preliminary model-building.

Skew: Statistic derived from the distribution of electron density values in an experimentally phased map. The skew describes the deviation from a Gaussian distribution; a correct map will have a slight skew towards higher values. A skew above 0.2 is usually indicative of successful phasing.

SMILES string: a simple textual representation of molecular structure, originally developed by Daylight Chemical Information Systems and now in wide use by the chemistry community. Although SMILES strings do not encode coordinates, unlike PDB files they uniquely encode the connectivity of molecules, and can optionally even specify chirality. They are therefore one of the preferred input formats for eLBOW. As an example, the SMILES string for benzene is c1ccccc1.

Special positions: locations on crystallographic symmetry elements. For macro-molecular structures special positions are always on rotation axes (2, 3, 4, 6-fold rotations); for small-molecule structures special positions may also be on centers of inversions, mirror planes, and roto-inversions. Atoms on special positions are restricted in the way they can move during refinement. For example an atom on a 2-fold axis can only move along the axis. It is also possible that atoms or molecules are disordered around special positions; in this case the occupancy found in the PDB file is usually 1/N or smaller, where N is the order of the rotational symmetry.

Above: an example of atoms on special positions in a potassium channel structure (PDB ID 1p7b <<http://www.rcsb.org/pdb/explore/explore.do?structureId=1P7B>>_). The potassium ions (grey spheres) lie on a two-fold crystallographic symmetry axis, with the asymmetric unit and symmetry mate shown as yellow and magenta lines respectively.

Structure factors: the individual reflections corresponding to the Fourier transform of the electron density of the crystal. Both $F(\text{obs})$ and $F(\text{model})$ are examples of structure factors, based on the true and model-based electron density, respectively. The term encompasses both the amplitudes and phases, although for the experimental structure factors, only the amplitudes can be measured directly. The amplitude alone is usually represented by the "abs()" operator.

Substructure: in the context of experimental phasing, the substructure consists of heavier elements whose sites can be identified based on their anomalous scattering (or in isomorphous replacement experiments, this may be done using the isomorphous differences between native and derivative datasets), and subsequently used to phase the entire structure. For instance, this might be selenium atoms in SeMet residues, endogenously bound metal ions, heavy metals or halide atoms soaked into the crystal, or just the native sulfur

or phosphorous atoms in proteins and nucleic acids. (In some contexts the term "substructure" may be used interchangeably with "heavy atom sites".) However, only those atoms with measurable scattering are important for phasing purposes. Depending on crystal contents and experiment type, the useful substructure can be anywhere from a single atom to hundreds of atoms. In Phenix, and in particular the AutoSol wizard, the substructure is initially identified by HySS and extended by Phaser.

Target function: in the context of refinement, this is the function that assesses the fit to experimental data (usually amplitudes). Older programs used least-squares (LS) targets, but modern refinement programs usually use maximum-likelihood (ML) targets, which are more robust in the presence of model error and incompleteness. In phenix.refine, the available target functions are ML, MLHL (amplitudes plus experimental phases), and LS (for refinement of twinned structures). Technically, any restraints used also specify a target function for minimization, although the term is not usually used in this context.

Target weights: since the experimental target function is normally combined with the geometry or B-factor restraints, a scale factor is applied. This can be automatically determined by comparing the gradients of each function, or by a grid search of empirically estimated values.

Translation function (TF): the second step of molecular replacement, which starts from the output of the rotation function and determines the exact position of the search model within the unit cell. The TF may be significantly more time-consuming than the RF, because a separate translation search must be run for each possible orientation indicated by the RF, but it can resolve space-group ambiguity, and the resulting Z-score (TFZ), in combination with the LLG, will usually be sufficient to determine whether the solution is correct or not.

Translational non-crystallographic symmetry (TNCS): a common crystal pathology in which the unit cell size is effectively doubled due to a near-symmetric translation operator which breaks higher symmetry. The effect of this on data is to double the number of reflections, with the additional reflections systematically weaker. TNCS can be detected by this signature, and by the presence of a large off-origin peak in the Patterson map; Xtriage is used in Phenix for this purpose. Although not as severe as twinning, TNCS can cause problems during phasing and (to a lesser extent) refinement, because it breaks some of the assumptions used in the maximum-likelihood equations used in Phenix. This is now handled by Phaser for molecular replacement and SAD phasing. See Zwart et al. (2008) *Acta Cryst D*. 64:99-107 for more information.

Translation, Libration, and Screw (TLS): A way of describing anisotropic motion for rigid groups of atoms which move collectively in the crystal. These are usually separate chains, or domains in a flexible protein, but there is no restriction on the size of groups. Refining TLS parameters will result in all atoms that are part of a TLS group being treated as anisotropic - in this sense, it is essentially constrained anisotropic refinement. Because it only adds twenty parameters per group, TLS refinement is suitable for almost any resolution. (Note that while phenix.refine allows you to perform either TLS or anisotropic refinement for separate atom selections within a single run, the two methods may not be combined, as they effectively refine the same parameters.)

Twin law: an operator describing the relationship between distinct reflections due to twinning. Each lattice has a limited set of permitted twin laws, which can be identified by Xtriage. These are typically represented in terms of Miller indices, for example, the R3 space group has the allowed merohedral twinning operator $-h-k,k,-l$, which signifies that the h,k,l reflection will be actually be overlapped in reciprocal space with $-h-k,k,-l$. The twin law can be used during refinement if necessary, although some caution should be taken when doing this. In rare cases a crystal can be a multiple twin, with more than one twin law. Note that phenix.refine does not currently support multiple twin laws.

Twinning: a common crystal pathology in which different regions of the crystal assume different orientations. In the simplest case, epitaxial twinning, this is simply two or more crystals stacked together, without

superimposed lattices, and the result is multiple distinct lattices visible in the diffraction pattern, which can often be handled by data processing programs. Merohedral twinning preserves the overall lattice symmetry, so that the diffraction spots contain the contribution of two or more "twin domains" which cannot easily be disambiguated. Small twin fractions (perhaps 10% or less) are not usually problematic and can easily be overlooked. Larger fractions, up to 50% ("perfect twinning"), can make experimental phasing very difficult, and result in abnormally high R-factors and poor map quality during refinement. This can be handled by refining with a twin law, although this needs to be performed with caution. Twinning is usually easily detected by the systematic deviation from expected intensity statistics; in Phenix, Xtriage is the program used to diagnose and analyze possible twinning. See Zwart et al. (2008) *Acta Cryst D*. 64:99-107 for more information.

Wilson plot: a plot showing the average intensity value for each resolution bin (typically using a relatively large number of bins, e.g. 30). Among other things, the Wilson plot shows the falloff of intensity with resolution due to the B-factors of the atoms, and can be used to determine an approximate overall B-factor for the data. Because of the non-random distribution of atoms in the unit cell, the Wilson plot has a distinctive appearance, especially for protein and nucleic acid structures, and deviations from the expected plot may indicate data pathologies.

A typical Wilson plot for a protein dataset (PDB ID 3dnd).

References

On macromolecular refinement at subatomic resolution with interatomic scatterers. P.V. Afonine, R.W. Grosse-Kunstleve, P.D. Adams, V.Y. Lunin, and A. Urzhumtsev. *Acta Crystallogr D Biol Crystallogr* 63, 1194-7 (2007).

- On macromolecular refinement at subatomic resolution with interatomic scatterers. P.V. Afonine, R.W. Grosse-Kunstleve, P.D. Adams, V.Y. Lunin, and A. Urzhumtsev. *Acta Crystallogr D Biol Crystallogr* 63, 1194-7 (2007).

Crystallographic R factor refinement by molecular dynamics. A.T. Brünger, J. Kuriyan, and M. Karplus. *Science* 235, 458-60 (1987).

- Crystallographic R factor refinement by molecular dynamics. A.T. Brünger, J. Kuriyan, and M. Karplus. *Science* 235, 458-60 (1987).

Free R value: a novel statistical quantity for assessing the accuracy of crystal structures. A.T. Brünger. *Nature* 355, 472-5 (2008).

- Free R value: a novel statistical quantity for assessing the accuracy of crystal structures. A.T. Brünger. *Nature* 355, 472-5 (2008).

Model bias in macromolecular crystal structures. A. Hodel, S.H. Kim, and A.T. Brunger. *Acta Cryst.* A48, 851-858 (1992).

- Model bias in macromolecular crystal structures. A. Hodel, S.H. Kim, and A.T. Brunger. *Acta Cryst.* A48, 851-858 (1992).

The other 90% of the protein: assessment beyond the Calphas for CASP8 template-based and high-accuracy models. D.A. Keedy, C.J. Williams, J.J. Headd, W.B. Arendall, V.B. Chen, G.J. Kapral, R.A. Gillespie, J.N. Block, A. Zemla, D.C. Richardson, and J.S. Richardson. *Proteins* 77 Suppl 9, 29-49 (2009).

- The other 90% of the protein: assessment beyond the Calphas for CASP8 template-based and high-accuracy models. D.A. Keedy, C.J. Williams, J.J. Headd, W.B. Arendall, V.B. Chen, G.J. Kapral, R.A. Gillespie, J.N. Block, A. Zemla, D.C. Richardson, and J.S. Richardson. *Proteins* 77 Suppl 9, 29-49 (2009).

Liking likelihood. A.J. McCoy. *Acta Crystallogr D Biol Crystallogr* 60, 2169-83 (2004).

- Liking likelihood. A.J. McCoy. *Acta Crystallogr D Biol Crystallogr* 60, 2169-83 (2004).

Averaged kick maps: less noise, more signal... and probably less bias. J. Pražnikar, P.V. Afonine, G. Guncar, P.D. Adams, and D. Turk. *Acta Crystallogr D Biol Crystallogr* 65, 921-31 (2009).

- Averaged kick maps: less noise, more signal... and probably less bias. J. Pražnikar, P.V. Afonine, G. Guncar, P.D. Adams, and D. Turk. *Acta Crystallogr D Biol Crystallogr* 65, 921-31 (2009).

Stereochemistry of polypeptide chain configurations. G.N. Ramachandran, C. Ramakrishnan, and V. Sasisekharan. *J Mol Biol* 7, 95-9 (1963).

- Stereochemistry of polypeptide chain configurations. G.N. Ramachandran, C. Ramakrishnan, and V. Sasisekharan. *J Mol Biol* 7, 95-9 (1963).

Improved Fourier coefficients for maps using phases from partial structures with errors. R.J. Read. *Acta Crystallographica* 42, 140-149 (1986).

- Improved Fourier coefficients for maps using phases from partial structures with errors. R.J. Read. *Acta Crystallographica* 42, 140-149 (1986).

Surprises and pitfalls arising from (pseudo)symmetry. P.H. Zwart, R.W. Grosse-Kunstleve, A.A. Lebedev, G.N. Murshudov, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 64, 99-107 (2008).

- Surprises and pitfalls arising from (pseudo)symmetry. P.H. Zwart, R.W. Grosse-Kunstleve, A.A. Lebedev, G.N. Murshudov, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 64, 99-107 (2008).

Building the PHENIX documentation

doc_procedures.html

To add a new page:

First make sure that \$PHENIX/phenix and \$PHENIX/phenix_html are up-to-date, and that phenix_html is checked out of SVN. Create a document, new_doc.txt, in phenix_html/rst_files. Add the file to the SVN. Add a reference to new_doc.html in phenix_documentation.html. execute: phenix_html.rebuild_docs

- First make sure that \$PHENIX/phenix and \$PHENIX/phenix_html are up-to-date, and that phenix_html is checked out of SVN.
- Create a document, new_doc.txt, in phenix_html/rst_files
- Add the file to the SVN
- Add a reference to new_doc.html in phenix_documentation.html
- execute: phenix_html.rebuild_docs

To update an existing document, just edit the corresponding file in phenix_html/rst_files, and then execute phenix_html.rebuild_docs.

Some steps expect that the environment variables PHENIX, PHENIX_VERSION, and PHENIX_RELEASE_TAG be defined. If you are working out of a Phenix installation updated to use SVN, this should not be a problem. Note that you may need to run 'libtbx.configure phenix_html' in the build directory for the command to appear on your path.

After running the update, the new HTML files will be available in \$PHENIX/doc, and may be viewed by running 'phenix.doc' (or clicking the help button in the PHENIX GUI).

Including PHIL parameter listing

If you want to have your documentation page show a full listing of program options with help text, simply place a directive like this at the end of the restructured text file:

```
{ { phil:phenix.command_line.autobuild } }
```

This gives the path to the module containing the PHIL object from which to generate a full keyword listing. Several different standard names will be checked when importing this object.

Including citations

Published references are stored in PHIL format in phenix/phenix/utilities/citations.params; this is not terribly different from BiBTeX format. Each publication is tagged with a simple ID (this could simply be the PubMed ID, but we usually prefer a program name or other human-readable identifier). In the documentation, full references may be automatically inserted (usually at the end of a document) by specifying something like:

```
{ { citation:molprobit } }
```

We encourage the use of this method for listing references, as it allows them to be easily incorporated into other documents (or displayed as part of program output, in the GUI, etc.).

A very fast introduction to restructured text format

The official documentation is here:

For Phenix documentation we use a relatively simple subset of features. The best way to learn the syntax is by examining files in the `rst_files` subdirectory (and comparing to the final HTML), but a simple example is shown below:

```
=====
This is a title
=====
```

```
Section title
-----
```

```
This is unformatted text; paragraphs are separated by blank lines. Use
**double-asterisk blocks** for bold text, and *single-asterisk blocks* for
italic text. Hyperlinks use the convention `link name &lt;url&gt;`_ (all special
characters are literal here).
```

- This is a bullet point!
- This is another bullet point

```
Another section title
-----
```

```
References section
-----
```

```
List of all available keywords
-----
```

You can add at the end some references as described above and the special link above that will create a keywords list.

PHENIX Examples

examples.html

Where can I find sample data?

You can find sample data in the directories located in: \$PHENIX/examples. Additionally there is sample MR data in \$PHENIX/phaser/tutorial. The PHENIX GUI provides the ability to automatically set up any of the tutorial datasets. When the GUI is started for the first time, a dialog window will appear asking you to create a project:

Click the button "Set up tutorial data" to select a tutorial. A new window will appear with a drop-down menu listing all of the tutorial datasets grouped by category (Experimental phasing, Molecular replacement, etc.). By default the necessary files will be copied to a new directory in your Documents folder (if present, otherwise the home directory will be used), but you may use any destination you wish.

A README file is available for some projects; this contains important information such as the wavelengths and anomalous scattering coefficients (f' and f'') for MAD datasets.

Can I easily run a Wizard with some sample data?

You can run sample data from the command line with a Wizard with a simple command. To run p9-sad sample data with the AutoSol wizard, you type:

```
phenix.run_example p9-sad
```

This command copies the \$PHENIX/examples/p9-sad directory to your working directory and executes the commands in the file run.sh. Documentation for running tutorials in the GUI is currently in preparation.

What sample data are available to run automatically?

You can see which sample data are set up to run automatically by typing:

```
phenix.run_example --help
```

This command lists all the directories in \$PHENIX/examples/ that have a command file run.sh ready to use. For example:

```
phenix.run_example --help
```

```
PHENIX run_example script. Fri Jul 6 12:07:08 MDT 2007
```

```
Use: phenix.run_example example_name [--all] [--overwrite]
Data will be copied from PHENIX examples into subdirectories
of this working directory
If --all is set then all examples will be run (takes a long time!)
If --overwrite is set then the script will overwrite subdirectories
```

```
List of available examples: 1J4R-ligand a2u-globulin-mr gene-5-mad
p9-build p9-sad
```

Are any of the sample datasets annotated?

The PHENIX tutorials listed on the main PHENIX web page will walk you through sample datasets, telling you what to look for in the output files. For example, the Tutorial 1: Solving a structure using SAD data tutorial uses the p9-sad dataset as example. It tells you how to run this example data in AutoSol and how to interpret

the results.

About Phenix-supported file formats

file_formats.html

Contents

- Model files
- Reflection files
- CIF files
- Sequence files
- MAP files
- PHIL files (.eff, .def, .phil)
- Geometry restraints info (.geo)
- Other chemical data (various)
- Log files (.log)

This document provides an introduction to the types of files used by (or generated by) Phenix. Many of these formats are also used by other software, but we have emphasized the most popular file types.

We do not recommend editing any of these file types by hand. Some may be human-readable (and log files, a special case, are intended almost entirely for this purpose), but in most cases the format is strictly defined and careless editing may lead to errors.

Model files

Models are represented primarily as a series of records for each atom, plus associated metadata such as crystal symmetry. The most common format in macromolecular crystallography is the PDB format, which might look like this:

```
ATOM 1 CA AILE A 1 9.968 51.572 -0.800 1.00 20.69 N
ATOM 2 CA BILE A 1 8.566 51.700 -0.272 1.00 21.30 C
HETATM 2545 ZN ZN A1317 36.762 44.115 -7.277 1.00 15.32 ZN
HETATM 2557 O HOH A2001 10.269 53.403 -3.264 1.00 27.45 O
```

Implicit in this information is a hierarchy of chains (with ID "A" in this case), residues (denoted by unique numbers), and atoms. Each residue may have multiple conformations (such as the isoleucine residue above, which has "A" and "B" conformers). The individual atom records have columns for the XYZ coordinates, occupancy, isotropic B-factor, and element symbol. (They also include a "serial number" - the first numerical column - but this is not usually used in Phenix.) If the structure is refined with anisotropic displacements (either for individual atoms, or TLS parameters), relevant atoms may be followed by an ANISOU record. Note that atoms may be denoted by either ATOM (for standard amino acid and nucleic acid residues) or HETATM (for waters, ions, ligands, and non-standard polymer residues). Most programs do not distinguish between these record types.

Besides atom information, the PDB format also usually includes a CRYST1 record specifying the unit cell and space group, and some number of REMARK records containing metadata such as refinement statistics, TLS matrices, and information about program use. The crystal symmetry is important for many applications, but the REMARK section is usually ignored (except for deposition with the PDB).

If you need to edit a PDB file (other than rebuilding the model in Coot), we recommend that you use either PDBTools, a non-interactive tool available both on the command line and the graphical interface, or

phenix.pdb_editor, an interactive graphical editor. Editing by hand is strongly discouraged and often leads to program errors.

In some programs in Phenix (in particular phenix.refine), support is also provided for reading and writing mmCIF files. This is a variation on CIF format (described in detail below) specific to macromolecular models and data. mmCIF is considerably more complicated than PDB format, and less human-readable, but it is also much more flexible and better-defined. In the future, all PDB depositions will be done using mmCIF format, but for now PDB files remain the primary model format during the structure determination process.

See the wwPDB site for full documentation on the PDB format and mmCIF.

Reflection files

Experimental data may be stored in a variety of formats. Some of these may include additional information, described below. In each format, a list of "Miller indices" (h, k, l) is associated with one or more data fields, but the exact contents vary widely.

Currently, the best-supported format in Phenix and other macromolecular crystallography software is MTZ format, which was developed as part of the CCP4 project. These are binary files (not human-readable), which makes them especially compact and fast. MTZ files may contain any number and combination of experimental datasets, R-free flags, phases, weights, or map coefficients. Each field has a unique column label; in Phenix, these are usually grouped together, for instance "F(+),SIGF(+),F(-),SIGF(-)" groups four columns from an MTZ file specifying the Friedel mates for experimental amplitudes and estimated error, and "2FOFCWT,PH2FOFCWT" specifies the amplitudes and phases for a 2mFo-DFC map. MTZ files will always explicitly specify the crystal symmetry, and include a field for the wavelength of each dataset (although this may not always be reliable).

In Phenix, every program that reads or writes reflection data supports MTZ format; these files should usually be compatible with other software such as CCP4 or Coot. (Note however that the wavelength output by Phenix programs may be incorrect, but this is not widely used.) Nearly any MTZ file is suitable as input, except for the raw output of Mosflm, which contains intensities that have not yet been scaled.

Several other formats are supported in various contexts. Some of these differ in whether individual observations of each unique reflection have been merged or not (see the separate documentation on using unmerged data in Phenix for more details), but all of these should contain scaled data. Only CNS format can include R-free flags; the rest are usually the output of data processing programs and require additional processing to be useful, and will ultimately be converted to MTZ format.

CNS format is text, and is the primary input and output format of the program of the same name. It may include any combination of data, but usually this will be amplitudes or intensities plus R-free flags. A variety of file extensions may be used, including .hkl, .fobs, or .cv.

Scalepack format is a text format output by the data processing software HKL2000. It only includes experimental intensities, with or without Friedel pairs. Crystal symmetry is explicitly supplied. These almost always use the extension .sca.

Unmerged Scalepack format is also produced by HKL2000, but includes unmerged intensities and additional data processing information. This is used by several programs (including SOLVE), but it has the disadvantage of lacking the unit cell parameters.

SHELX format is an even simpler text format. We do not recommend using this if possible, because it doesn't explicitly specify either the crystal symmetry or the data type (amplitudes or intensities). These files also tend to have the extension .hkl.

XDS format is also text, specifying experimental intensities and associated information from data processing. XDS actually outputs several different types of files, but the most appropriate will nearly always be named "XDS_ASCII.HKL". These files contain crystal symmetry information and may include either merged or unmerged data.

If you need to edit or combine reflection files, Phenix includes a graphical editor which supports all of these formats. However, output is limited to MTZ. Visualization of reflection data is provided by `phenix.data_viewer`.

CIF files

CIF (Crystallographic Information Format) is a general-purpose machine-readable syntax for storing any type of crystallographic information. The contents of a CIF file are defined by specific "tags", many of which are officially defined by a committee. A typical CIF file might look like this (simplified from the PDB's chemical components database):

```
data_ALA
#
 _chem_comp.id ALA
 _chem_comp.name ALANINE
 _chem_comp.formula "C3 H7 N O2"
 loop_
 _chem_comp_bond.comp_id
 _chem_comp_bond.atom_id_1
 _chem_comp_bond.atom_id_2
 _chem_comp_bond.value_order
 ALA N CA SING
 ALA N H SING
```

In small molecule crystallography CIF is used widely, but in Phenix the primary use is for geometry restraints used in refinement and related tasks. The Phenix installer includes a large number of CIF files including some from the PDB and the CCP4 monomer library, but for novel and/or uncommon ligands you may have to generate your own restraints using `eLBOW`. You can also edit an existing restraints CIF file using `REEL`.

In addition to geometry restraints, both models and reflection data may be stored in CIF format using the mmCIF subset of tags. These are the preferred working format for the PDB, which only distributes structure factors in mmCIF format. A variety of tools are able to convert to PDB or MTZ formats; the most commonly used program for this purpose is `phenix.cif_as_mtz`. Some programs such as `phenix.refine` can also read and write mmCIF files.

Sequence files

A number of programs require sequence information as input, including Phaser, AutoSol and AutoBuild. Several different formats are allowed, all of which contain the sequence as single-character codes (usually uppercase). For a single sequence, no other information needs to be specified. Otherwise, the most common format is FASTA, which can contain multiple sequences with a header for each, for instance:

```
> lysozyme
KVFGRCELAAAMKRHGLDNYRGYSLGNWVCAAKFESNFNTQATNRNTDGSTDYGILQINSRWWCNDGRTPGSRN
LCNIPCSALLSSDITASVNC AKKIVSDGNGMNAWVAWRNRCKGTDVQAWIRGCRL
> rnase
KETAAAKFERQHMDSSSTAASSSNYCNQMMKSRNLTKDRCKPVNTFVHESLADVQAVCSQKNVACKNGQTNCYQ
SYSTMSITDCRETGSSKYPNCAYKTTQANKHIIIVACEGNPYVPVHFDAVS
```

An alternative way to represent multiple sequences is to separate each sequence with a blank line:

```
KVFGRCELAAAMKRHGLDNYRGYSLGNWVCAAKFESNFNTQATNRNTDGSTDYGILQINSRWWCNDGRTP
GSRN LCNIPCSALLSSDITASVNC AKKIVSDGNGMNAWVAWRNRCKGTDVQAWIRGCRL KETAAAKFERQ
```

HMDSSTSAASSSNYCNQMMKSRNLTKDRCKPVNTFVHESLADVQAVCSQKNVACKNGQTNCYQ
SYSTMSITDCRETGSSKYPNCAYKTTQANKHIIVACEGNPYVPVHFDASV

KVFGRCELAAAMKRHGLDNRYGYS LGNWVCAAKFESNFNTQATNRNTDGSTDY GILQINSRWWCNDGRTP
GSRN LCNIPCSALLSSDITASVNC AKKIVSDGNGMNAWVAWRNRCKGTDVQAWIRGCRL

KETAAAKFERQHMDSSTSAASSSNYCNQMMKSRNLTKDRCKPVNTFVHESLADVQAVCSQKNVACKNGQT
NCYQ SYSTMSITDCRETGSSKYPNCAYKTTQANKHIIVACEGNPYVPVHFDASV

MAP files

Although maps are usually stored as the Fourier coefficients (weighted amplitude and phase), which is more convenient and saves disk space, several programs allow them to be output in CCP4 or X-plor format. These are then suitable for viewing in PyMOL, Chimera, or other molecular viewers (as well as Coot). We recommend that you use CCP4 format, which is binary and therefore much faster and smaller than the text X-PLOR format files. If you have map coefficients for which you would like to obtain a map file, phenix.mtz2map will convert them for you by running an FFT. In the Phenix GUI, most programs that output map coefficients will automatically perform the FFT if you click a button to open the results in PyMOL.

PHIL files (.eff, .def, .phil)

PHIL, which stands for "Python Hierarchial Input Language", is the standard format for specifying program parameters in Phenix and CCTBX. It uses a very lightweight syntax that is both human-readable and writeable. GUI users will not normally need to view or edit these files (although they are extensively used internally), but some knowledge of the syntax may be helpful anyway. A typical PHIL file might look like this (loosely based on the parameters used in phenix.refine):

```
refinement {  
  input {  
    pdb {  
      file_name = model.pdb  
      file_name = ligands.pdb  
    }  
  }  
  refine.strategy = *individual_sites *individual_adp tls occupancies  
  main {  
    ordered_solvent = True  
    number_of_macro_cycles = 10  
  }  
}
```

Some parameters or parameter groups ("scopes") can have multiple values, as in the case of refinement.input.pdb.file_name above. Consult the manual for each individual program for more information about specific options.

Geometry restraints info (.geo)

These files are generated by phenix.refine and related programs. They are not widely used but are very helpful for debugging issues with geometry restraints.

Other chemical data (various)

eLBOW supports a variety of other formats (in addition to CIF) for specifying chemical information to be used to generate molecular structures and restraints. The most popular is SMILES, but the output of various cheminformatics programs is also recognized.

Log files (.log)

Most log files do not follow any specific format, and vary widely between different programs. Even for GUI users, we recommend that you familiarize yourself with the log output as it may contain useful debugging information not displayed graphically, as well as providing some insight into how a program works.

Main PHENIX Modules

phenix-modules.html

Data Analysis

Detection of twinning and other pathologies is facilitated via the program `phenix.xtriage`. This command line driven program analyses an experimental data set and provides diagnostics that aid in the detection of common idiosyncrasies such as the presence of pseudo translational symmetry, certain data processing problems and twinning. Other sanity checks, such as a Wilson plot sanity check and an algorithm that tries to detect the presence of ice rings from the merged data are performed as well. If twin laws are present for the given unit cell and space group, a Britton plot is computed, an H-test is performed and a likelihood based method is used to provide an estimate of the twin fraction. Twin laws are deduced from first principles for each data set, avoiding the danger of over-looking twin laws by incomplete lookup tables. If a model is available, more efficient twin detection tools are available. The RvsR statistic is particularly useful in the detection of twinning in combination with pseudo rotational symmetry. This statistic is computed by `phenix.xtriage` if calculated data is supplied together with the observed data. A more direct test for the presence of twinning is by refinement of the twin fraction given an atomic model (which can be performed in `phenix.refine`). The command line utility `phenix.twin_map_utils` provides a quick way to refine a twin fraction given an atomic model and an X-ray data set and also produces

Automated Structure Solution Using Experimental Phasing Techniques

Structure solution via SAD, MAD or SIR(AS) can be carried out with the AutoSol wizard. AutoSol performs heavy atom location, phasing, density modification and initial model building in an automated manner. The heavy atoms are located with substructure solution engine also used in `phenix.hyss`, a dual space method similar to SHELXD and Shake and Bake. Phasing is carried out with PHASER for SAD cases and with SOLVE for MAD and SIR(AS) cases. Subsequent density modification is carried out with RESOLVE. The hand of the substructure is determined automatically on the basis of the quality of the resulting electron density map. It is noteworthy that the whole process is not necessarily linear but that the wizard can decide to step back and (for instance) try another set of heavy atoms if appropriate. In the resulting electron density map, a model is built (currently limited to proteins). Further model completion can be carried out via the AutoBuild wizard. AutoBuild iterates model building and density modification with refinement of the model in a scheme similar to other iterative model building methods, for example ARP/wARP.

Automated Structure Solution Via Molecular Replacement

Structure solution via molecular replacement is facilitated via the MRage procedure. MRage guides the user through setting up all necessary parameters to run a molecular replacement job with PHASER. The molecular replacement carried out by PHASER uses likelihood based scoring function, improving the sensitivity of the procedure and the ability to obtain reasonable solutions with search models that have a relatively low sequence similarity to the crystal structure being determined. Besides the use of likelihood based scoring functions, structure solution is enhanced by detailed bookkeeping of all search possibilities when searching for more than a single copy in the asymmetric unit or when there the choice of space group is ambiguous. When a suitable molecular replacement solution is found, the AutoBuild wizard is invoked and rebuilds the molecular replacement model given the sequence of the model under investigation.

Automated Model Building

Automated model building given a starting model or a set of reasonable phases can be carried out by the AutoBuild wizard. A typical AutoBuild job combines density modification, model building, macromolecular refinement and solvent model updates ('water picking') in an iterative manner. Various modes of building a model are available. Depending on the availability of a molecular model, model building can be carried by locally rebuilding an existing model (rebuild in place) or by building in the density without any information of an available model. The rebuilding in place model building is a powerful building scheme that is used by default for molecular replacement models that have a high sequence similarity to the sequence of the structure that is to be built. A fundamental feature of the AutoBuild wizard is that it builds various models, all from slightly different starting points. The dependency of the outcome of the model building algorithm on initial starting conditions provides a straightforward mechanism to obtain a variety of plausible molecular models. It is not uncommon that certain sections of a map are built in one model, while not in another. Combining these models allows the AutoBuild wizard to converge faster to a more complete model than when using a single model building pass for a given set of phases. Dedicated loop fitting algorithms are used to close gaps between chain segments. This feature, together with the water picking and side chain placement, typically results in highly complete models of high quality that need minimal manual intervention before they are ready for deposition.

Structure Refinement

The refinement engine used in the AutoBuild and AutoSol wizards can also be run from the command line with the phenix.refine command. The phenix.refine program carries out likelihood based refinement and has the possibility to refine positional parameters, individual or grouped atomic displacement parameters, individual or grouped occupancies. The refinement of anisotropic displacement parameters (individual or via a TLS parameterization) is also available. Positional parameters can be optimized using either traditional gradient-only based optimization methods, or via simulated annealing protocols. The command line interface allows the user to specify which part of the model should be refined in what manner. It is in principle possible to refine half of the molecule as a rigid group with grouped B values, whereas the other half of the molecule has a TLS parameterization. The flexibility of specifying the level of parameterization of the model is especially important for the refinement of low resolution data or when starting with severely incomplete atomic models. Another advantage of this flexibility in refinement strategy is that a user can perform a complex refinement protocol that carries out simulated annealing, isotropic B refinement and water picking in 'one go'. Another main feature of phenix.refine is the way in which the relative weights for the geometric and ADP restraints with respect to the X-ray target are determined. Considerable effort has been put into devising a good set of defaults and weight determination schemes that results in a good choice of parameters for the data set under investigation. Defaults can of course be overwritten if the user chooses to. Besides being able to handle refinement against X-ray data, phenix.refine can refine against neutron data or against X-ray and neutron data simultaneously.

Automated ligand density analysis

Automated fitting of ligands into the electron density is facilitated via the LigandFit wizard. The ligand building is performed by finding an initial fit for the largest rigid domain of the ligand and extending the remaining part of the ligand from this initial 'seed'. Besides being able to fit a known ligand into a difference map, the LigandFit wizard is capable of identifying ligands on the basis of the difference density only. In the latter scheme, density characteristics for ligands occurring frequently in the PDB are used to provide the user with a range of plausible ligands.

Calculating ligand geometries and defining chemical restraints

Stereo chemical dictionaries of ligands whose chemical description is not available in the supplied monomer library for the use in restrained macromolecular refinement can be generated with the electronic ligand builder and optimization workbench (eLBOW). eLBOW generates a 3D geometry from a number of chemical input formats including MOL2 or PDB files and SMILES strings. SMILES is a compact, chemically dense description of a molecule that contains all element and bonding information and optionally other stereo information such as chirality. To generate a 3D geometry from an input format that contains no 3D geometry information, eLBOW uses a Z-Matrix formalism in conjunction with a table of bond lengths calculated using the Hartree-Fock method with a 6-31G(d,p) basis set to obtain a Cartesian coordinate set. The geometry is then optionally optimized using the semi-empirical quantum chemistry method AM1. The AM1 optimization provides chemically meaningful and accurate geometries for the class of molecule typically complexed with proteins. eLBOW outputs the optimized geometry and a standard CIF restraint file that can be read in by phenix.refine and can also be used for real space refinement during manual model building sessions in the program COOT. An interface is also available to use eLBOW within COOT.

Literature

- Adams PD, Grosse-Kunstleve, R.W., and Brunger, A.T.: Computational aspects of high-throughput crystallographic macromolecular structure determination. *Methods Biochem Anal* 2003, 44:75-87.
- Terwilliger TC, Berendzen J: Automated MAD and MIR structure solution. *Acta Crystallogr D Biol Crystallogr* 1999, 55(Pt 4):849-861.
- Schneider TR, Sheldrick GM: Substructure solution with SHELXD. *Acta Crystallogr D Biol Crystallogr* 2002, 58(Pt 10 Pt 2):1772-1779.
- McCoy AJ, Grosse-Kunstleve RW, Storoni LC, Read RJ: Likelihood-enhanced fast translation functions. *Acta Crystallographica Section D* 2005, 61(4):458-464.
- Terwilliger TC: Automated main-chain model building by template matching and iterative fragment extension. *Acta Crystallogr D Biol Crystallogr* 2003, 59(Pt 1):38-44.
- Terwilliger TC: Automated side-chain model building and sequence assignment by template matching. *Acta Crystallogr D Biol Crystallogr* 2003, 59(Pt 1):45-49.
- Emsley P, Cowtan K: Coot: model-building tools for molecular graphics. *Acta Crystallogr D Biol Crystallogr* 2004, 60(Pt 12 Pt 1):2126-2132.
- Adams PD, Grosse-Kunstleve RW, Hung L-W, Ioerger TR, McCoy AJ, Moriarty NW, Read RJ, Sacchettini JC, Sauter NK, Terwilliger TC: PHENIX: building new software for automated crystallographic structure determination. *Acta Crystallographica Section D* 2002, 58(11):1948-1954.
- Grosse-Kunstleve RW, Sauter NK, Moriarty NW, Adams PD: The Computational Crystallography Toolbox: crystallographic algorithms in a reusable software framework. *Journal of Applied Crystallography* 2002, 35:126-136.
- Grosse-Kunstleve RW, Adams PD: Substructure search procedures for macromolecular structures. *Acta Crystallogr D Biol Crystallogr* 2003, 59(Pt 11):1966-1973.
- Weeks CM, Miller R: Optimizing Shake-and-Bake for proteins. *Acta Crystallogr D Biol Crystallogr* 1999, 55(Pt 2):492-500.
- Read R: Pushing the boundaries of molecular replacement with maximum likelihood. *Acta Crystallographica Section D* 2001, 57(10):1373-1382.
- Schomaker V, Trueblood K: On Rigid-Body Motion of Molecules in Crystals. *Acta Crystall B-Stru* 1968, B 24:63-&.

- Winn MD, Isupov MN, Murshudov GN: Use of TLS parameters to model anisotropic displacements in macromolecular refinement. *Acta Crystallogr D Biol Crystallogr* 2001, 57(Pt 1):122-133.
- Brunger AT, Adams PD, Rice LM: Annealing in crystallography: a powerful optimization tool. *Prog Biophys Mol Biol* 1999, 72(2):135-155.
- Vagin AA, Steiner RA, Lebedev AA, Potterton L, McNicholas S, Long F, Murshudov GN: REFMAC5 dictionary: organization of prior chemical knowledge and guidelines for its use. *Acta Crystallogr D Biol Crystallogr* 2004, 60(Pt 12 Pt 1):2184-2195.
- Weininger D: SMILES 1. Introduction and Endoding Rules. *J Chem Inf Comput Sci* 1988, 28:31.
- Fisher RG, Sweet RM: Treatment of diffraction data from crystals twinned by merohedry. *Acta Crystallographica Section A* 1980, 36(5):755-760.
- Yeates TO: Simple statistics for intensity data from twinned specimens. *Acta Crystallogr A* 1988, 44 (Pt 2):142-144.
- Yeates TO: Detecting and overcoming crystal twinning. *Methods Enzymol* 1997, 276:344-358.
- Lebedev AA, Vagin AA, Murshudov GN: Intensity statistics in twinned crystals with examples from the PDB. *Acta Crystallogr D Biol Crystallogr* 2006, 62(Pt 1):83-95.
- Zwart P: Anomalous signal indicators in protein crystallography. *Acta Crystallographica Section D* 2005, 61(11):1437-1448.

Additional information

The phenix graphical interface

phenix_gui.html

Contents

- Overview
- Getting help from the Phenix chatbot
- Running programs from the GUI
- Modules
- Using AlphaFold and other predicted models in Phenix
- Crystallography Data analysis and manipulation Validation and map-based comparisons Experimental phasing Molecular replacement Maps (create, manipulate, compare) Enhanced maps (Polder, FEM, density-modified...) Model building Refinement Ligands
- Data analysis and manipulation
- Validation and map-based comparisons
- Experimental phasing
- Molecular replacement
- Maps (create, manipulate, compare)
- Enhanced maps (Polder, FEM, density-modified...)
- Model building
- Refinement
- Ligands
- Cryo-EM Map analysis, symmetry, manipulation Map improvement Docking, model building and rebuilding Refinement
- Map analysis, symmetry, manipulation
- Map improvement
- Docking, model building and rebuilding
- Refinement
- Models Superpose, search, compare, analyze symmetry Modification, minimization and dynamics Other tools
- Superpose, search, compare, analyze symmetry
- Modification, minimization and dynamics
- Other tools
- Projects
- Selection Editor
- Running PHENIX tutorials
- Preferences
- Coot integration
- PyMOL integration

- Reporting bugs
- Acknowledgments References
- References

Overview

The Phenix GUI is primarily a frontend to the command-line programs, with several extra graphical utilities for validation, map generation, and file manipulations. The original GUI is still available but deprecated, and all functionality has been more or less superseded. This page covers the main interface and common behavior; individual program GUIs are covered separately. The GUI runs on all supported operating systems but with some limitations on specific platforms, in particular Microsoft Windows.

The main GUI is started simply by typing the command phenix; by default, it will also open automatically when you launch any of the individual program GUIs. You should not run more than one instance of the GUI at a time, to avoid conflicts with internal database files.

Getting help from the Phenix chatbot

You can use the Phenix chatbot to get help on using any Phenix program in the GUI. Just ask it any question about using Phenix. You can access the chatbot with the Chat button on the toolbar in the GUI.

Running programs from the GUI

When starting a job, Phenix writes out a configuration file and calls the command-line version of the program. The method of execution varies depending on operating system. On Linux and Windows, By default the job is started directly in the main process, i.e. "locally", which allows communication between the program and the GUI in memory rather than via temporary files. The drawback to this is that if the GUI is closed or crashes, the job will be ended too. An alternate "detached" mode is available (and used as default on Mac), which starts the job as an entirely separate process. This limits the speed at which the GUI can be updated, but allows quitting the GUI without stopping the job.

A third mode, available only to Linux users, is to run jobs on a queueing system; while this could be done entirely on a multi-core workstation, it will usually be spread across a cluster of similar computers. Currently Sun Grid Engine, PBS, LSF, and Condor are supported to varying degrees. To enable queued jobs, open the Preferences, switch to the "Processes" tab, and check the box to enable queueing. The queue submission option will now appear when starting a job. For this to work, the queue job management binaries (for instance, 'qsub', 'qstat', and 'qdel' in SGE) must be in the current environment \$PATH, Phenix must be installed in the same location on all nodes, and the filesystem on which the job is being run must be mounted on all nodes. The GUI will display a "waiting" status until the job is actually started. The main interface has a window for viewing the current queue status (Utilities->Show queue status).

Modules

Individual programs are grouped by category. Except where noted, most of these correspond to command-line programs, and the documentation for the command-line version should be the primary reference for understanding program behavior and inputs. Additional GUI documentation is available for some programs.

Using AlphaFold and other predicted models in Phenix

- Overview: Overview of AlphaFold and predicted models in Phenix, with links to video tutorials.
- PredictAndBuild: How to solve an X-ray or cryo-EM structure using AlphaFold models.
- PredictModel: How to predict a structure with AlphaFold using the Phenix server through the Phenix GUI.
- ProcessPredictedModel: How to process a predicted model and obtain just the high-confidence parts, split into domains.

Crystallography

Data analysis and manipulation

- Xtrriage: comprehensive reflection data analysis and quality assessment; used to detect twinning and other pathologies. Also used internally in the AutoSol and AutoBuild wizards.
- Calculate merging statistics: calculate R-sym, R-meas, mean I/sigma, CC1/2, and related statistics starting from scaled, unmerged intensities.
- Analyze anomalous signal in a SAD experiment
- Scale unmerged anomalous data or multiple datasets
- Reflection file editor: utility for merging reflection files and creating or extending R-free flags. (GUI only)
- Import CIF structure factors: convert a CIF file (used by the PDB to store deposited experimental data) to an MTZ file. This will automatically fetch data for a known ID from the PDB, or you may supply your own files.
- French & Wilson data correction: procedure for estimating appropriate values for weak and negative intensities. This is also run automatically as part of several programs, including phenix.refine and the Phaser GUIs.
- 3D and 2D data viewers: these display data from reflection files (usually amplitudes or intensities) as they appear in reciprocal space, either in a full 3D view or a pseudo-precession camera section.

Validation and map-based comparisons

- Comprehensive validation: based on the MolProbity server (and sharing much of the same code), with added analysis of experimental data. Reports R-work and R-free, statistics for geometry restraints, Ramachandran plot, sidechain rotamers, C-beta deviation, all-atom contacts, and real-space correlation with electron density. Outlier lists are linked to graphics programs such as Coot, and clicking a residue or atom will zoom in on that site in the graphics window. Coot will also display clashes detected by PROBE.
- Structure comparison: a tool for evaluating assorted model features and validation criteria for multiple related structures, and highlighting regions of difference. Automatically superposes model and maps into a common frame of reference for viewing in Coot or PyMOL.
- Comparison of unmerged data quality with refined model, as described in Karplus & Diederichs (2012). Phenix.cc_star.
- EMRinger: Model validation for de novo electron microscopy structures
- Map and model-map correlations and offsets: Map-map and map-model correlation allowing optional translational offsets
- POLYGON: graphical comparison of user-selected model statistics with similar structures in the PDB.

Experimental phasing

- AutoSol: automated experimental phasing for all experiment types (SAD, MAD, MIR, etc.); also performs simple model-building after phasing. Combines HySS, Phaser, SOLVE, RESOLVE, and phenix.refine.
- Hybrid substructure search: heavy-atom site identification program. Used automatically as part of AutoSol.
- Phaser-EP: Another Phaser interface, for SAD and MR-SAD phasing. We recommend trying AutoSol first, but this GUI exposes additional parameters.
- Plan a SAD experiment: Estimate required data quality for successful SAD phasing given experimental parameters

Molecular replacement

- MRage: automated molecular replacement using Phaser;
- Phaser-MR: Interface for molecular replacement using standalone Phaser, with all parameters available as well as different modes of operation. We recommend starting with MRage first, but this GUI is useful for tough cases.
- MR-Rosetta: automation pipeline for exceptionally difficult structures, which uses the Rosetta software for protein structure prediction and design to rebuild poor MR solutions, along with Phaser and AutoBuild. (Separate installation of Rosetta is required.)
- Sculptor: prepare a search model for molecular replacement by trimming the structure, modifying B-factors, etc.
- Sculptor - Coot interface: a Coot plugin for running Sculptor interactively and visualizing results.
- Ensembler: tool for creating superimposed ensembles of related MR search models.

Maps (create, manipulate, compare)

- Create map coefficients: simple GUI for generating likelihood-weighted maps, including 2mFo-DFc, mFo-DFc, anomalous, and others, including averaged "kick" maps. Currently works only on untwinned data. (GUI only)
- Generate composite omit maps: Tool for creating a composite omit map.
- Isomorphous difference map: simple utility for creating a map from two sets of amplitudes. Corresponds to the command-line program phenix.fobs_minus_fobs_map.
- Cut out density: extracts a section of electron density from input map coefficients, and outputs new map coefficients (suitable for running molecular replacement in Phaser).
- Superpose maps: given two PDB files and accompanying map coefficients (in MTZ format), superposes the PDB files and reorients the maps to follow them. Output is the reoriented PDB files and maps in CCP4 format.
- Map sigma level comparison: Calculate equivalent sigma levels when visually comparing two maps
- Find difference map peaks and holes: Identify local maxima and minima in mFo-DFc map (and anomalous map if available) and flag waters with excess density
- Model-based phases: calculate phases and structure factors in a variety of formats (including Hendrickson-Lattman coefficients).

- Create a map from map coefficients: convert an MTZ file containing map coefficients to one or more CCP4 or XPLOR-format maps suitable for viewing in PyMOL, etc. (Note that the main maps GUI and many other programs can also output these files, and the GUI will generate them for you as necessary for PyMOL.)
- Optimize a map by sharpening: Automatic or manual map sharpening/blurring.
- Calculate F(model): utility for generating structure factors (as real or complex numbers) from a model alone.

Enhanced maps (Polder, FEM, density-modified...)

- Calculate Polder maps: compute omit map excluding bulk solvent polder.html
- RESOLVE density modification: This is just a simplified interface for running the AutoBuild wizard in map-only mode.
- Solvent flipping density modification: Simple density modification by solvent flipping
- Multi-crystal averaging: performs density modification on different crystal forms of the same structure and averages the maps around the models.
- Maximum entropy map: Statistical map modification procedure to remove artifacts due to missing data
- Feature Enhanced Map (FEM): Calculate a 2mFo-DFc map locally scaled and density-modified to enhance fine details

Model building

- PredictAndBuild: Solve an X-ray structure using AlphaFold models.
- AutoBuild: automated model-building and refinement with RESOLVE and phenix.refine. This GUI only encapsulates the building functions; a separate GUI is available for omit-map calculations.
- Phase and build: similar to AutoBuild, but significantly faster (although less accurate).
- Find Helices and Strands: fast building of secondary structure (as poly-ALA) into maps with RESOLVE and PULCHRA. Especially good for low-resolution data.
- Morph model: improvement of poor molecular replacement solutions by "morphing" into density and rebuilding.
- Map to model: Model-building into cryo-EM and low-resolution maps

Refinement

- phenix.refine: automated refinement, supporting both X-ray and neutron data. In addition to the features available in the command-line program, the GUI version includes graphical atom selection, simplified setup of restraints, automatic addition of hydrogens, and post-refinement validation.
- phenix.real_space_refine: automated refinement using a map and model (CCP4 format or MTZ format with map coefficients)
- DEN refinement: Deformable elastic network refinement using simulated annealing, for low-resolution and molecular replacement structures
- Ensemble refinement: Time-averaged molecular dynamics refinement - models disorder using a combination of multiple models and TLS
- Rosetta refinement: Hybrid Rosetta/Phenix refinement for low-resolution X-ray crystal structures

- ReadySet: preparation of input files for refinement, including generating restraints (CIF) files and adding hydrogens.
- AmberPrep: Utility for preparing files for refinement automatically generate the four files needed for Phenix-Amber

Ligands

- LigandFit: wizard for placing ligands in electron density maps, accounting for necessary conformational changes.
- eLBOW: electronic Ligand Builder and Optimization Workbench, a tool for generating restraints (and geometries) for any molecule, using a variety of inputs including PDB file and SMILES string.
- Ligand pipeline: Automated molecular replacement, refinement, and ligand placement for high-throughput crystallography
- REEL: "Restraints Editor Especially Ligands", a graphical editor for CIF files, also serves as a frontend to eLBOW. Currently runs as a separate program. (GUI only)
- Ligand search: program for identifying unknown blobs of electron density and placing appropriate ligand(s).
- Guided ligand replacement: Ligand fitting based on an existing protein-ligand complex

Cryo-EM

Map analysis, symmetry, manipulation

- Mtriage: Analyze quality of cryo-EM maps and models
- Local resolution: Calculate a local resolution map
- Map symmetry: Find map reconstruction symmetry
- Map box: Extract box of density with model or extract unique part

Map improvement

- Optimize a map by sharpening: Automatic or manual map sharpening/blurring.
- Resolve Cryo EM: Density modify a cryo EM map
- Combine Focused Maps: Align maps and combine the best parts of each

Docking, model building and rebuilding

- PredictAndBuild: Interpret a cryo-EM map using AlphaFold models.
- Dock in map: Dock a model into map
- Map to model: Model-building into cryo-EM maps
- Pathwalker: Chain tracing with cryo-EM maps
- Rebuild model: Rebuild a model, keeping connectivity
- Sequence from map: Determine a sequence from a map
- Fit Loops: Build missing loops into a map based on a sequence and starting model
- Douse (Add waters): Add waters to cryo-EM models

- CryoFit: Flexibly fit a model to a cryo-EM map
- phenix.real_space_refine: automated refinement using a map and model (CCP4 format or MTZ format with map coefficients)
- eLBOW: electronic Ligand Builder and Optimization Workbench, a tool for generating restraints (and geometries) for any molecule, using a variety of inputs including PDB file and SMILES string.

Models

Superpose, search, compare, analyze symmetry

- Superpose models: structure superposition program.
- PDB Search: Match phi-psi angles or sequence: Ramachandran or sequence-based PDB search
- Secondary structure PDB search: SSM match, morph and trim
- Fragment search: Find and morph matching fragments from PDB
- Chain comparison: Find similarity between two main-chains
- Replace with fragments from PDB: Replace entire model with fragments from PDB
- NCS identification: determine appropriate NCS operators and/or restraint groups based on a set of heavy-atom sites, macromolecule chains, or electron density.
- Find alternate-origin symmetry mate: Transform a model to place it in same ASU and frame of reference as another

Modification, minimization and dynamics

- Process predicted model: Replace B-factor field in model and optionally split into domains
- Trim overlapping: Trim parts of a model overlapping another model
- PDB Tools: various model manipulations such as modifying atom records, geometry minimization, and generating "fake" structure factors.
- PDB editor: graphical tool for editing PDB files
- Apply NCS: apply NCS operators to a PDB file to generate the complete asymmetric unit. Note that a simpler NCS GUI is also available as a plugin to the phenix.refine GUI, and does not necessarily need to be run separately.
- Model Reconstruction: Get asymmetric unit or biological unit with BIOMT or MTRX
- Geometry minimization: Regularize geometry for a model (without data)
- Simple dynamics: Shake a model using crude molecular dynamics
- Combine PDB files: simple utility for merging models split over several PDB files (such as massive complexes like the ribosome). Chain IDs will be modified if necessary (using up to two characters).
- Add conformations: Add alternate conformations in bulk, for an entire model or user-defined atom selection
- Perturb model with normal mode analysis: Uses NMA in Phaser to generate perturbed models along the direction of the normal modes
- Identify domains with normal mode analysis (SCEDS): Uses NMA in Phaser to identify approximate rigid domains

- Sort heteroatom groups: Rearrange the non-polymer heteroatom groups in a model to group with the nearest macromolecule chain, similar to the protocol used by the PDB
- Prepare model for PDB deposition: Finalize mmCIF files for deposition to the PDB
- Get PDB validation report: Retrieve a validation report from the PDB
- Generate Table 1: utility for extracting statistics from PDB, reflection, and log files required for publication.
- Find a Program: Search for a program or test

Other tools

A number of simpler tasks involving file modification and visualization can be accessed by clicking the button labeled "Other tools" on the toolbar.

Current options include:

- Fetch PDB data: Download structure data in PDB, FASTA, or CIF/MTZ format. (Command line equivalent: `phenix.fetch_pdb.`)
- Convert PDB file(s) to FASTA: Extract sequences of polymer chains in a model, with flexible options for handling gaps, insertions, etc. (Command line equivalent: `iotbx.pdb_as_fasta.`)
- Multiple sequence alignment: uses the public-domain program MUSCLE (Edgar 2005) to perform alignment of sequences in various formats (including automatic extraction of sequences from PDB files). (Command line equivalent: none, but MUSCLE can be run in PHENIX with the command `phenix.muscle.`)
- Plot average B-factors in PDB file: Plots residue B-factors (for all atoms, backbone, or sidechain) as a horizontal line graph. (Command line equivalent: `mmtbx.bfactor_plot.`)
- Reindex reflections: Performs change-of-basis operations on reflection files. (Command-line equivalent: `phenix.reindex.`)
- Modify PDB file: Shortcuts for performing some of the most common tasks in `phenix.pdbtools` (also available as a full GUI).
- PDB remediation: Convert between versions 2 and 3 of the PDB format. (Command-line equivalent: `iotbx.pdb_remediator.`)
- Import data and flags: Prepares data for use in refinement; this tool is especially useful when bringing in new high-resolution reflections for an already-refined structure.
- Plot PDB statistics: Generates scatter plots or histograms of statistics output by `phenix.model_vs_data` for a large fraction of the PDB (nearly all X-ray structures with experimental data available). The same database is used to generate the plots in POLYGON.
- Inspect R-free flags: Plots distribution of R-free flags in a reflections file; this is primarily used to determine whether the flags have been created in thin resolution shells. (Command line equivalent: `wxtbx.inspect_r_free_flags.`)

Projects

Like the CCP4 GUI (`ccp4i`), Phenix manages data and job history by grouping into projects. You will be prompted to create a project the first time you start the GUI. On subsequent launches Phenix will attempt to guess the project based on the current directory. There are several constraints on project management:

Project IDs should be no more than 24 characters and contain only alphanumeric characters (including underscore). Project directories may not be nested, i.e. you may not create a project for a directory that is part

of an existing project, or that has a subdirectory that is owned by another project.

- Project IDs should be no more than 24 characters and contain only alphanumeric characters (including underscore).
- Project directories may not be nested, i.e. you may not create a project for a directory that is part of an existing project, or that has a subdirectory that is owned by another project.

When a project is created Phenix will create a folder ".phenix" in the project directory; this is used to store job history, temporary files, and other internal data. Users should not need to modify this folder unless deleting the project. All functions related to project management are available from the main GUI only, either in the toolbar or the File menu.

The current version of the GUI has limited functionality for configuring projects. Several options are available for setting the behavior of phenix.refine, such as default parameter files. The GUI will give you the opportunity to create these files automatically, but this feature is optional. The other primary function of the projects is storing job history; although this feature is still in development, it will save basic statistics and configuration files, and can restore the parameters and results of any successful previous run.

You may switch between projects while a GUI program is running; however, each program window stays associated with the project it was opened with.

Selection Editor

A unified interface for making atom selections. More details can be found on this page.

Running PHENIX tutorials

A number of examples using published structures are included in the PHENIX installation, and can be automatically loaded into the GUI as new projects. To load tutorial data, click on "New project" in the main GUI; this window also appears the first time the PHENIX GUI is started (before any projects have been created). The button "Set up tutorial data" will open a new dialog with a list of examples, grouped by the methods they are intended to demonstrate.

For live demonstration, we normally use the P. aerophilum translation initiation-factor 5a or p9-sad example for experimental phasing, TEM-1 Beta-lactamase/beta-lactamase inhibitor complex (beta-blip) for molecular replacement, S. aureofaciens ribonuclease Sa (rnase-s) for refinement, and N-ethylmaleimide sensitive factor + ATP (nsf-d2-ligand) for ligand fitting. All of these datasets run relatively quickly (5-30 minutes) in the intended programs. (These examples are also used extensively in this documentation.)

Preferences

Some of the behavior of the GUI can be customized via the Preferences dialog, which is available from the File menu (Linux) or Phenix menu (Mac) and on the toolbar of most programs.

Because some of the larger programs may have up to 500 distinct parameters, many of which rarely need changing, advanced settings are hidden by default. You may control this in individual dialogs using the "User level" menu at the bottom of the window, or change the global level in the "PHENIX interface" pane (shown above), in the "User level" option. For example, in phenix.refine, a typical configuration dialog will look like this when the user level is "Basic":

When set to "Advanced", many more options will appear:

Other Preferences settings determine what external programs are used for various file types, interactions with molecular graphics programs (Coot, PyMOL, and the simple built-in graphics), and options for some of the specific modules in Phenix.

Coot integration

Coot is an open-source (GPL) model-building program written by Paul Emsley. Although we do not distribute it with Phenix, it is available as source and binaries for Linux (from the developer's page), and third-party Mac binaries are available (provided by Bill Scott). Documentation on how to use it with Phenix is here. Phenix will try to locate Coot on your system automatically, but if it is not found, you may specify the command to use under Preferences->Graphics->Full path to Coot.

PyMOL integration

PyMOL is an open-source molecular viewer written by Warren DeLano. We distribute an older precompiled version (0.99) with Phenix; however, we recommend installing the latest version and add the command under Preferences->Graphics->Full path to PyMOL. More information is in the separate PyMOL instructions.

Reporting bugs

Bugs in the code may result in a pop-up window containing a detailed error message. Clicking "OK" will send an email to the Phenix developers with this information. We strongly encourage users to submit these reports, as they are one of the primary mechanisms by which we identify coding errors. The report will include the error message, information about the host system, and user name/email; no other personally identifying information or data is sent.

Acknowledgments

The PHENIX GUI is written in Python 2.7, using the wxPython toolkit for most features, plus matplotlib for plotting. Some of the icons used in the GUI came from the Crystal Icons project by Everaldo Coelho which is licensed under the LGPL. Many additional icons (anything with a molecular representation) were generated using PyMOL.

Several open-source programs have been included with PHENIX and are used in various ways in the GUI; we are grateful to their authors for permission to redistribute the code:

****MUSCLE**** written by Bob Edgar. Edgar, R.C. (2004) MUSCLE: multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Res.* 32(5):1792-1797. ****ksDSSP**** written by UCSF Computer Graphics Laboratory. Original method (independently reimplemented): W. Kabsch and C. Sander, "Dictionary of Protein Secondary Structure: Pattern Recognition of Hydrogen-Bonded and Geometrical Features" *Biopolymers* 22:2577 (1983). PULCHRA (Rotkiewicz & Skolnick 2008)

- ****MUSCLE**** written by Bob Edgar. Edgar, R.C. (2004) MUSCLE: multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Res.* 32(5):1792-1797.
- ****ksDSSP**** written by UCSF Computer Graphics Laboratory. Original method (independently reimplemented): W. Kabsch and C. Sander, "Dictionary of Protein Secondary Structure: Pattern Recognition of Hydrogen-Bonded and Geometrical Features" *Biopolymers* 22:2577 (1983).
- PULCHRA (Rotkiewicz & Skolnick 2008)

References

Graphical tools for macromolecular crystallography in PHENIX. N. Echols, R.W. Grosse-Kunstleve, P.V. Afonine, G. Bunkóczy, V.B. Chen, J.J. Headd, A.J. McCoy, N.W. Moriarty, R.J. Read, D.C. Richardson, J.S. Richardson, T.C. Terwilliger, and P.D. Adams. *J. Appl. Cryst.* 45, 581-586 (2012). Fast procedure for reconstruction of full-atom protein models from reduced representations. P. Rotkiewicz, and J. Skolnick. *J Comput Chem* 29, 1460-5 (2008).

- Graphical tools for macromolecular crystallography in PHENIX. N. Echols, R.W. Grosse-Kunstleve, P.V. Afonine, G. Bunkóczy, V.B. Chen, J.J. Headd, A.J. McCoy, N.W. Moriarty, R.J. Read, D.C. Richardson, J.S. Richardson, T.C. Terwilliger, and P.D. Adams. *J. Appl. Cryst.* 45, 581-586 (2012).
- Fast procedure for reconstruction of full-atom protein models from reduced representations. P. Rotkiewicz, and J. Skolnick. *J Comput Chem* 29, 1460-5 (2008).

Overview of Phenix Programs

phenix_programs.html

Contents

- General
- Assessing data quality
- Experimental Phasing
- Molecular Replacement
- AlphaFold and other predicted models
- Model Building
- Structure Refinement
- Validation
- Ligands
- Deposition
- Reflection data tools
- Map tools
- Model tools
- Miscellaneous

General

- phenix: Run Phenix graphical user interface
- phenix.about: Summarize contributors, packages, and info for phenix
- phenix.acknowledgments: Summarize third-party components of Phenix
- phenix.commands: List command line Phenix methods
- phenix.doc: Phenix documentation
- phenix.help: Load Phenix documentation (same as phenix.doc)
- phenix.version: Print version of Phenix
- phenix.find_program: Find a program or regression test
- phenix.list: Print list of Phenix commands
- phenix_regression.list: Print list of Phenix regression commands

Assessing data quality

- phenix.cc_star: Calculate cc_star values for correlation of data vs resolution
- phenix.compare_datasets: Similar to phenix.data_viewer, as side-by-side view
- phenix.data_viewer: View pseudo-precession planes through a dataset
- phenix.emma: Compare heavy-atom solutions
- phenix.explore_metric_symmetry: Compare unit cells

- `phenix.merging_statistics`: Calculate statistics on unmerged data
- `phenix.xtriage`: Analyze data files for quality and unusual conditions

Experimental Phasing

- `phenix.autosol`: Automated structure solution by MR/MAD/SAD
- `phenix.find_ncs`: Find NCS in a model, map or heavy-atom sites
- `phenix.hyss`: Identify heavy-atom sites from anomalous dataset
- `phenix.density_modification`: Density modification
- `phenix.multi_crystal_average`: Multi-crystal averaging
- `phenix.ncs_and_number_of_ha`: Guess solvent content and number of heavy-atom sites
- `phenix.ncs_average`: NCS average (no density modification) and write map file
- `phenix.phase_and_build`: Rapid density modification and model-building

Molecular Replacement

- `phenix.automr`: Automated MR and model-building
- `phenix.cut_out_density`: Create MTZ coeffs with density cut out from a map
- `phenix.find_alt_orig_sym_mate`: Superimpose structures allowing origin shifts (see also `map_to_object`)
- `phenix.morph_model`: Morph a model to match a map
- `phenix.mr_model_preparation`: Download and edit PDB files for MR
- `phenix.mr_rescoring`: Model scoring for `mr_rosetta`
- `phenix.mr_rosetta`: MR and model improvement with phaser/autobuild/Rosetta
- `phenix.mr_rosetta_rebuild`: Rebuild model with Rosetta
- `phenix.phaser`: Run PHASER
- `phenix.sculptor`: Improve molecular replacement models using sequence alignment and structural information
- `phenix.ensampler`: Superpose PDB files to create ensemble for MR
- `phenix.sculpt_ensemble`: Combined sculptor and ensampler

AlphaFold and other predicted models

- `phenix.predict_and_build`: Solve X-ray structure or interpret cryo-EM map with iterative prediction and rebuilding
- `phenix.process_predicted_model`: Process AlphaFold2 and other predicted models
- `phenix.dock_predicted_model`: Dock processed AlphaFold2 models into cryo-EM map
- `phenix.dock_and_rebuild`: Process, dock and rebuild AlphaFold2 model into cryo-EM map

Model Building

- `phenix.assign_sequence`: Assign sequence to a chain using a map and seq file
- `phenix.autobuild`: Iterative model-building density modification and refinement

- `phenix.build_one_model`: Build one model using a map and data file
- `phenix.build_rna_helices`: Build RNA helices into a map
- `phenix.combine_models`: Take best parts of two models
- `phenix.dock_and_rebuild`: Process, dock and rebuild AlphaFold2 model into cryo-EM map
- `phenix.dock_predicted_model`: Dock processed AlphaFold2 models into cryo-EM map
- `phenix.find_helices_strands`: Build helices and strands into a map
- `phenix.fit_loops`: Fit missing loops in a model
- `phenix.map_to_model`: Build a model using a low-resolution map or cryo-EM map
- `phenix.trace_and_build`: Build a model by chain tracing using a cryo-EM map
- `phenix.douse`: Build water into cryo-EM map

Structure Refinement

- `phenix.den_refine`: Carry out DEN refinement of a model
- `phenix.ensemble_refinement`: run ensemble refinement
- `phenix.find_tls_groups`: Automatic identification of appropriate TLS groups in model
- `phenix.ready_set`: Set up files for refinement, including addition of hydrogens, generation of ligand restraints, and metal coordination restraints
- `phenix.real_space_refine`: Extensive real-space refinement
- `phenix.refine`: Carry out refinement of a model
- `phenix.secondary_structure_restraints`: generate pseudo H-bond restraints for alpha helices, beta sheets, and nucleic acid base pairs

Validation

- `phenix.b_factor_statistics`: Display summary of atomic displacement parameters for a model (or atom selection)
- `phenix.cablam_training`: C-alpha-based protein secondary structure exploration
- `phenix.cablam_validate`: C-alpha-based protein secondary structure exploration
- `phenix.cbetadev`: Validate protein sidechain C-beta geometry
- `phenix.clashscore`: Evaluate model based on all-atom contacts
- `phenix.kinemage`: Generates a multi-criterion validation kinemage file, for viewing in KiNG
- `phenix.king`: Run KiNG molecular graphics
- `phenix.map_correlations`: Compare maps or map and model, optionally allowing translational offsets
- `phenix.model_vs_data`: Evaluate model using experimental data
- `phenix.molprobity`: Run molprobity
- `phenix.probe`: Run PROBE, for analysis of all-atom contacts
- `phenix.r_factor_statistics`: Distribution of Rfree, Rwork and Rfree-Rwork for PDB models at similar resolution
- `phenix.ramalyze`: Validate protein backbone Ramachandran dihedral angles
- `phenix.reduce`: Run REDUCE, software for addition or trimming of hydrogens

- `phenix.rna_validate`: Validate RNA sugar puckers, backbone bond and angle geometry, and backbone suite conformations
- `phenix.rotalyze`: Validate protein sidechain rotamers
- `phenix.holton_geometry_validation` : Assess overall geometry

Ligands

- `phenix.elbow`: Ligand builder (CIF from PDB, SMILES etc)
- `phenix.find_all_ligands`: Find ligands from a list in a map
- `phenix.ligand_identification`: Identify ligands from a map
- `phenix.ligand_pipeline`: Automated molecular replacement, refinement, and ligand fitting for high-throughput crystallography
- `phenix.ligandfit`: Fit ligands into a map
- `phenix.metal_coordination`: Generate metal coordination bond and angle restraints
- `phenix.reel`: Graphical ligand restraints editor

Deposition

- `mmtbx.prepare_pdb_deposition`: produce mmCIF files for deposition
- `phenix.get_pdb_validation_report`: get validation report from RCSB via OneDep API

Reflection data tools

- `phenix.cif_as_mtz`: Convert CIF to MTZ
- `phenix.cif_as_pdb`: Convert CIF to PDB format
- `phenix.cns_as_mtz`: Convert CNS to MTZ
- `phenix.csv_as_mtz`: Convert csv reflection files to MTZ
- `phenix.fmodel`: Calculate structure factors from model
- `phenix.import_and_add_free`: Import a data file and add Free R set
- `phenix.mtz.dump`: Dump MTZ file contents
- `phenix.mtz_as_cif`: Convert mtz to CIF format
- `phenix.reciprocal_space_arrays`: Create MTZ file with Fmodel, Fcalc, Fbulk, Fmask, FOM, HL, resolution and more
- `phenix.reflection_file_converter`: Basic conversion between reflection file formats (command-line)
- `phenix.reflection_statistics`: Evaluation and comparison of reflection data (e.g. anomalous difference correlations)
- `phenix.reindex`: Reindex an MTZ file
- `phenix.remove_aniso`: Remove anisotropy from columns of an MTZ dataset
- `phenix.xmanip`: Experimental tool for manipulation of reflection data

Map tools

- `phenix.average_map_coeffs`: Average a set of map coefficients from several files

- `phenix.composite_omit_map`: Generate composite omit map, with or without refinement and simulated annealing
- `phenix.density_outside_model`: Remove density near a model from a map
- `phenix.cut_out_density`: Create MTZ coeffs with density cut out from a map
- `phenix.segment_and_split_map`: Split a map into regions of density
- `phenix.map_sharpening`: Optimize a map by sharpening
- `phenix.local_resolution`: Calculate a local resolution map
- `phenix.dock_in_map`: Dock a model into a map
- `phenix.map_symmetry`: Identify symmetry in a map
- `phenix.resolve_cryo_em`: Density modify a cryo-EM map
- `phenix.map_model_correlations`: Map and map-model correlation allowing translations
- `phenix.guess_molecular_centers`: Guess molecular centers from local RMS density
- `phenix.map_box`: Simple cut out map around a PDB file
- `phenix.map_to_model_histogram`: Compute averaged radial density distribution
- `phenix.map_to_object`: Superimpose using SG symmetry only (see also `find_alt_orig_sym_mate`)
- `phenix.map_to_structure_factors`: Calculate structure factors and HL coefficients from ccp4 map file and save in MTZ
- `phenix.map_value_at_point`: Get map value at given coordinates
- `phenix.maps`: Create maps from PDB and MTZ files
- `phenix.maximum_entropy_map`: Compute maximum entropy map from map coefficients
- `phenix.polder`: Compute omit map which excludes the bulk solvent around the omitted region
- `phenix.remove_free_from_map`: Set all map coeffs of free reflections to zero
- `phenix.resolve_cryo_em`: Density modify a cryo-EM map
- `phenix.superpose_maps`: Superimpose PDB files and transform map to match
- `phenix.mtriage`: Tool to analyse cryo-EM map (resolution, etc)

Model tools

- `phenix.apply_ncs`: Apply NCS (.ncs_spec file) to a chain to create molecule
- `phenix.dynamics`: Shake up structure with simple molecular dynamics
- `phenix.form_factor_query`: f' and f'' table lookup given element and wavelength
- `phenix.fragment_search`: Find fragments in PDB matching a target fragment
- `phenix.ksdssp`: Identify secondary structure in a model
- `phenix.model_model_distances`: Distance between two PDB files: per atom, residue, chain, model and overall
- `phenix.muscle`: sequence alignment tool
- `phenix.pdb.hierarchy`: Quick summary of PDB file content
- `phenix.pdb_as_cif`: Convert PDB format to CIF

- `phenix.pdb_atom_selection`: Extract selected atoms from PDB file (useful for experimenting with atom selections)
- `phenix.pdb_editor`: Edit PDB files graphically
- `phenix.pdb_interpretation`: Read PDB file and build restraints for refinement (useful for trouble-shooting)
- `phenix.pdbtools`: Manipulate PDB files
- `phenix.perigee`: Interaction finder
- `phenix.print_sequence`: Print sequence from PDB file
- `phenix.process_predicted_model`: Process AlphaFold2 and other predicted models
- `phenix.pulchra`: PULCHRA conversion from CA to full chain
- `phenix.replace_with_fragments_from_pdb`: Replace model with fragments from the PDB
- `phenix.search_and_morph`: Find Structures in PDB using SSM matching and morphing
- `phenix.simple_ncs_from_pdb`: NCS from a PDB file (use instead `phenix.find_ncs`)
- `phenix.structure_search`: Find Similar Structures in PDB using phi-psi angles and sequence
- `phenix.superpose_ligands`: Superimpose two ligands
- `phenix.superpose_models`: Superimpose models using aligned sequences or SSM
- `phenix.trim_overlapping`: Remove residues in a PDB file overlapping another
- `phenix.trim_pdb`: Remove hydrogen atoms from a PDB file

Miscellaneous

- `phenix.default_gui_settings`: Print out base PHENIX GUI configuration (mostly useful for site admins)
- `phenix.get_latest_version`: Download most recent installer replacing current installation
- `phenix.grow_density`: Density modification to enhance chain ends
- `phenix.pymol`: Pymol
- `phenix.python`: Run phenix-cognizant version of python
- `phenix.resolve`: Run resolve
- `phenix.resolve_pattern`: run resolve_pattern
- `phenix.run_example`: Run an example from the `phenix_examples` directory
- `phenix.show_build_path`: Show path to Phenix build directory
- `phenix.show_dist_paths`: Show paths to all components of Phenix
- `phenix.solve`: Run SOLVE
- `phenix.start_coot`: Coot molecular graphics
- `phenix.where_mon_lib_list_cif`: Show location of monomer library used by Phenix

Using PyMOL with PHENIX

pymol.html

Author(s)

PyMOL was originally written by Warren DeLano, and is now distributed by Schrodinger LLC.

- PyMOL was originally written by Warren DeLano, and is now distributed by Schrodinger LLC.

Instructions

NOTE: We distribute an obsolete build of PyMOL (version 0.99) with the Linux installers for PHENIX; however, we recommend using a more recent version for more advanced functionality such as anisotropic B-factor ellipsoids and object groups. There are several options for obtaining PyMOL: you can purchase a license from Schrodinger, you can download and compile the latest open-source code yourself, or you may be able to install it automatically using a third-party packaging and distribution system if one exists for your platform. (Macintosh users can use Fink.) The extensions provided by PHENIX do not require any features beyond those in open-source PyMOL, but the full documentation (and the native Mac GUI) is only available with a license.

Configuring and launching PyMOL from within the PHENIX GUI is very similar to what is done for Coot; if it is already part of your PATH environment variable or installed in a standard location such as /Applications on Macintosh, no additional configuration will be necessary. Most programs in PHENIX which output models and/or maps will have buttons to automatically open these in PyMOL. Although PyMOL is limited to reading pre-calculated map files (not map coefficients in MTZ format), PHENIX will perform the necessary FFT when requested to generate CCP4 maps if not are available.

Most applications output either a simple F(obs) map (FP PHI FOM), often after density modification, or a pair of 2mFo-DFc and mFo-DFc difference maps. By default, the F(obs) and 2mFo-DFc maps will be contoured at 1 sigma (1 standard deviation above the mean) and colored blue, and the mFo-DFc map will be contoured at +/- 3 sigma and colored green and red. You can change these colors in the Preferences (Color section).

Like Coot, PyMOL can be used as a viewer for any PDB file loaded in the PHENIX GUI by clicking the magnifying glass icon next to the input controls or file name. You can make it the default viewer in the Preferences (Graphics section).

Setting up your view in PyMOL

Here are some simple controls that let you choose what you see in PyMOL, assuming that you have a model (pdb_1) and a map (map_1) or maps loaded.

Click a few times on "pdb_1" you will see the model turn off and on. Same for "contour_1.5". Similarly "all" turns everything on and off, and "map_1" turns the unit cell box (which may not be visible in your viewer) on and off. To the right of "pdb_1" you will see buttons labelled "A" "S" "H" "L" and "C". Click on each one and you'll see what they do: - "A" : Actions. lets you recenter, delete the object, and more - "S" : Show. For a model you may want to show sticks for clarity. - "H" : Hide. Undoes show. - "L" : Label. Choose what labels to display - "C" : Color. Choose colors. The little table in the lower right of the PyMOL display window shows what each mouse button does. If you have a 3-button (2 buttons and a roller) mouse, then hold the left button down and move the mouse to rotate; right button down and move the mouse to change the size; both buttons down and move the mouse to move the center. If you accidentally click the wrong buttons and some new

object appears on the screen that you do not want, click on the "A" button for the new object and select "delete" to get rid of it.

- Click a few times on "pdb_1" you will see the model turn off and on. Same for "contour_1.5". Similarly "all" turns everything on and off, and "map_1" turns the unit cell box (which may not be visible in your viewer) on and off.
- To the right of "pdb_1" you will see buttons labelled "A" "S" "H" "L" and "C". Click on each one and you'll see what they do: - "A" : Actions. lets you recenter, delete the object, and more - "S" : Show. For a model you may want to show sticks for clarity. - "H" : Hide. Undoes show. - "L" : Label. Choose what labels to display - "C" : Color. Choose colors.
- The little table in the lower right of the PyMOL display window shows what each mouse button does. If you have a 3-button (2 buttons and a roller) mouse, then hold the left button down and move the mouse to rotate; right button down and move the mouse to change the size; both buttons down and move the mouse to move the center.
- If you accidentally click the wrong buttons and some new object appears on the screen that you do not want, click on the "A" button for the new object and select "delete" to get rid of it.

Additional resources

PyMOL home page PyMOL Wiki, an excellent community-supported resource.

- PyMOL home page
- PyMOL Wiki, an excellent community-supported resource.